Git & GitHub

© All rights reserved. www.keepcoding.io

¿Qué es Git y para qué sirve?

- Un sistema de control de versiones
- Un software de apoyo para desarrollar
- Control sobre la evolución de un proyecto
- Control sobre el desarrollo colaborativo
- Desarrollo en paralelo de funcionalidades
- Estructuración y mantenimiento de versiones

© All rights reserved. www.keepcoding.io

¿Cómo se instala?

- En Windows, usando el instalador
- En macOs, usando homebrew, MacPorts o instalando XCode
- En Linux, usando el instalador de paquetes de tu distribución

© All rights reserved. www.keepcoding.io

Configurando Git

git config

```
# Mostrar el valor del parámetro de configuración <param>
git config <param>

# Establece el valor <value> en el parámetro de configuración <param>
git config <param> <value>

# Establece el valor para cualquier proyecto Git en este ordenador
git config --global <param> <value>

# Ejemplos:

# Establecer nuestro nombre
git config --global user.name "Homer J. Simpson"

# Establecer nuestro e-mail (no tiene por qué ser el de GitHub)
git config --global user.name "Homer J. Simpson"
```

© All rights reserved. www.keepcoding.io

Nuestro primer repo

`git init`

```
# Creamos una carpeta para nuestro proyecto
mkdir star-wars

# Accedemos a la carpeta del proyecto
cd star-wars

# Inicializamos nuestro repo
git init --initial-branch=main
```

Un repo de git es una carpeta de nuestro ordenador en la que hemos ejecutado el comando **`git init`**. Este comando crea una carpeta oculta llamada **`.git`** donde git almacena toda su info.

© All rights reserved. www.keepcoding.io

Las tres zonas

Working Copy	Staging Area	Graph (mentira)
• Es la carpeta de nuestro proyecto.	• Es “el lugar” donde pondremos los cambios que queremos guardar una versión.	• Es un “lugar virtual” donde se almacenan los cambios.
• Podemos ver lo que hay en su interior con el explorador de archivos.	• Sólo podemos ver lo que hay en su interior usando algún comando de *git*.	• Sólo podemos ver lo que hay en su interior usando algún comando de *git*.
• También se le conoce como *Working Tree* o *Unstaged Files*.	• También se conoce como *index* o *cache*.	• Su nombre real es *repositorio,* pero este nombre nos puede confundir.

© All rights reserved. www.keepcoding.io

Flujo de trabajo habitual

Working Copy → **Staging Area** → **Graph (mentira)**

Escribimos código (hacemos cambios) usando nuestra herramienta favorita.

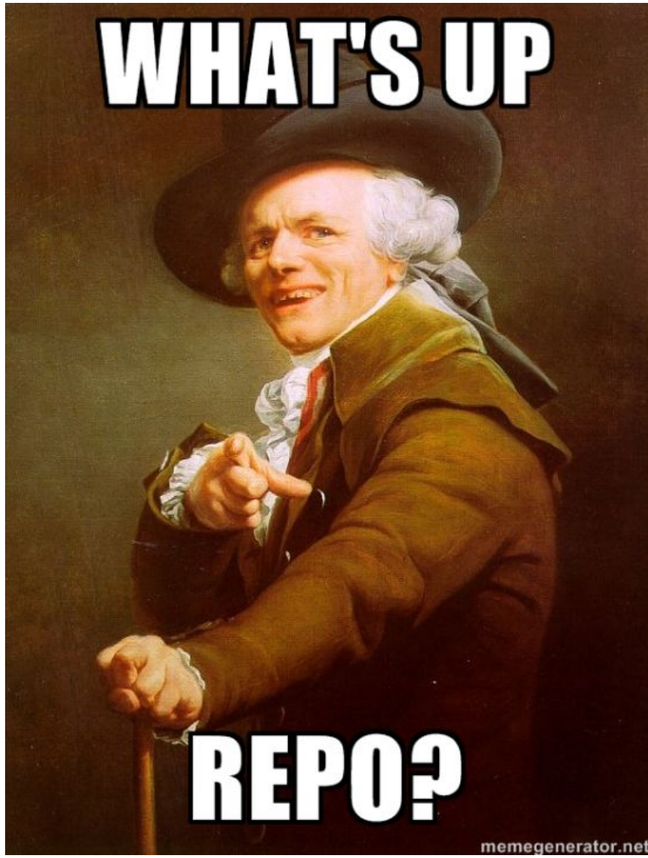
De todo el código que hemos escrito, seleccionamos los archivos cambiados de los que queremos guardar una versión.

Guardamos los cambios que se han puesto en Staging Area

© All rights reserved. www.keepcoding.io

`git status`

- El comando `git status` nos resume el estado del proyecto.
- Nos dice qué archivos no tienen ninguna versión guardada.
- Qué archivos han cambiado desde su última versión guardada.
- Qué archivos están en el staging área y cuáles no.



© All rights reserved. www.keepcoding.io

git add: poniendo cambios en el Staging Area

Working Copy → Staging Area

git add

```
# Añade al Staging Area los cambios del archivo <file>
git add <file>

# Añade al Staging Area todos los archivos con cambios dentro de la carpeta <folder>
git add <folder>

# Añade al Staging Area todos los archivos con cambios que terminen por .md
git add *.md
```

© All rights reserved. www.keepcoding.io

git commit: del Staging Area al Graph

Staging Area → Graph (mentira)

git commit

```
# Hace un commit con los cambios que hay en Staging Area
# Antes, abrirá un editor de texto para que podamos introducir el mensaje
git commit

# Hace un commit con los cambios que hay en Staging Area con el mensaje indicado
git commit -m "Mensaje del commit"
```

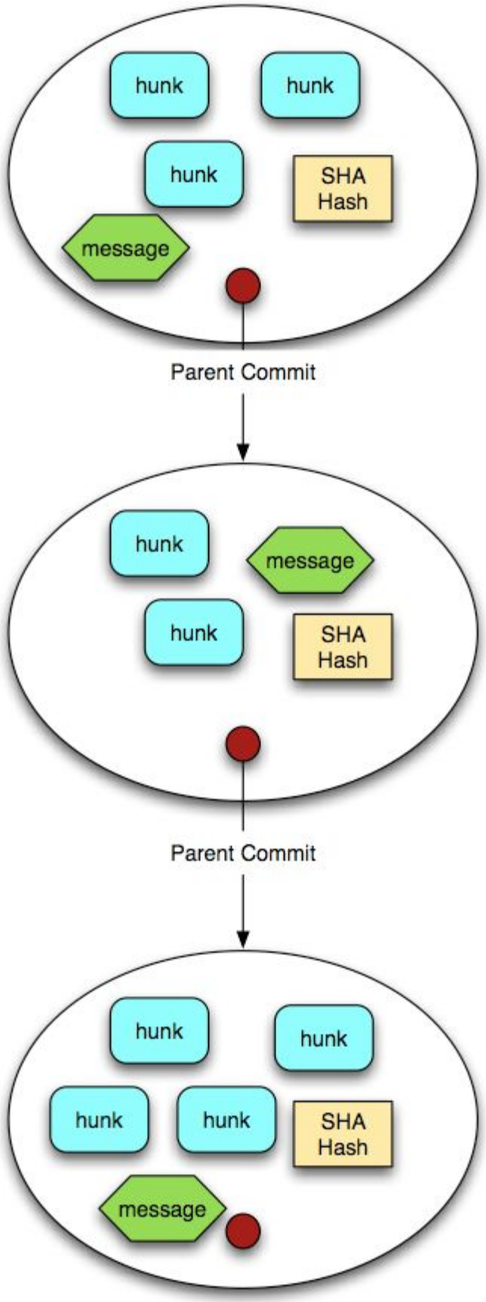
© All rights reserved. www.keepcoding.io

¿Qué es un commit?

Un **commit es un paquete** que contiene:

- Uno o más hunks (cambios en archivos)
- Un mensaje que describe qué cambios
- Un hash SHA para identificar el commit (un identificador único)
- La fecha y hora de creación
- El nombre y e-mail del autor
- **Un enlace con su "commit padre"**

© All rights reserved. www.keepcoding.io



© All rights reserved. www.keepcoding.io

¿Qué pasa en **Graph** cuando hago un commit?

© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 🧐

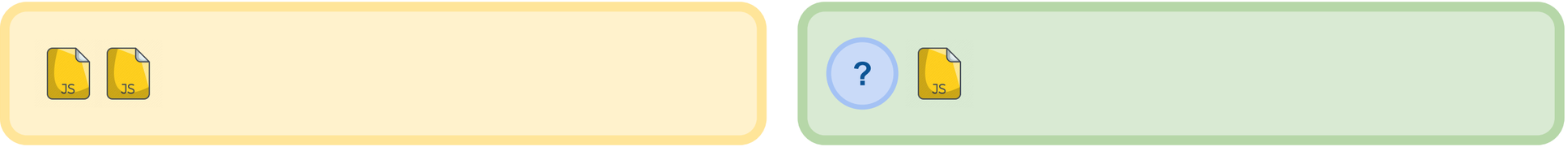
© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 🧐

```
git add *.js
```

© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 🧐



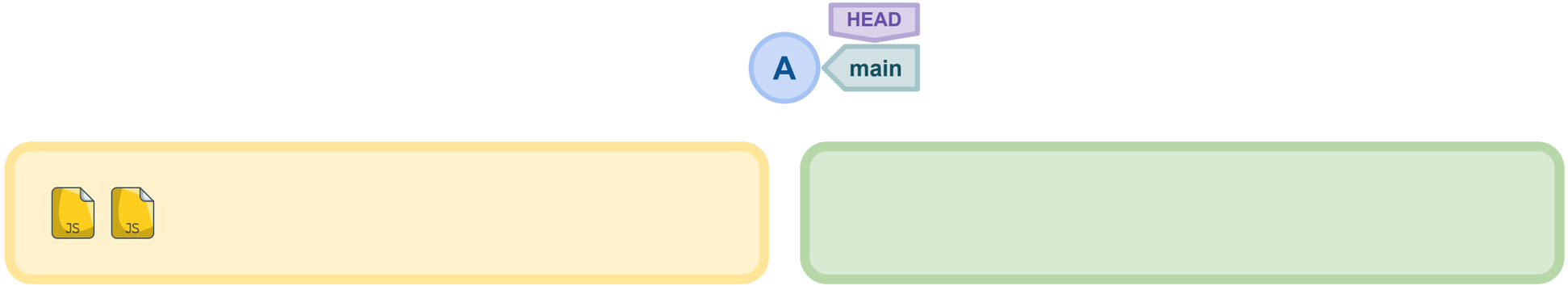
© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 🧐

```
git commit -m “Mi primer commit”
```

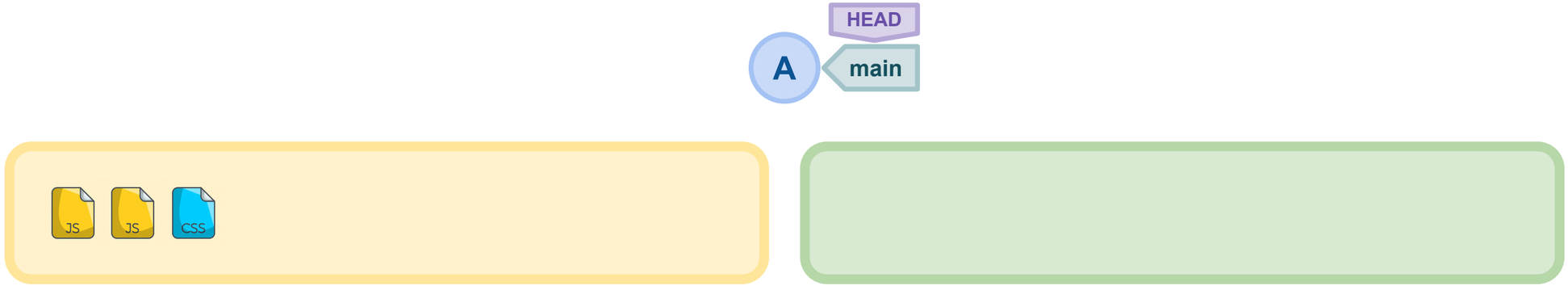
© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 🧐



© All rights reserved. www.keepcoding.io

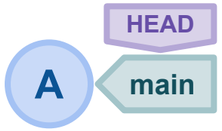
Git Simulator v.1.3.73 🧐



© All rights reserved. www.keepcoding.io

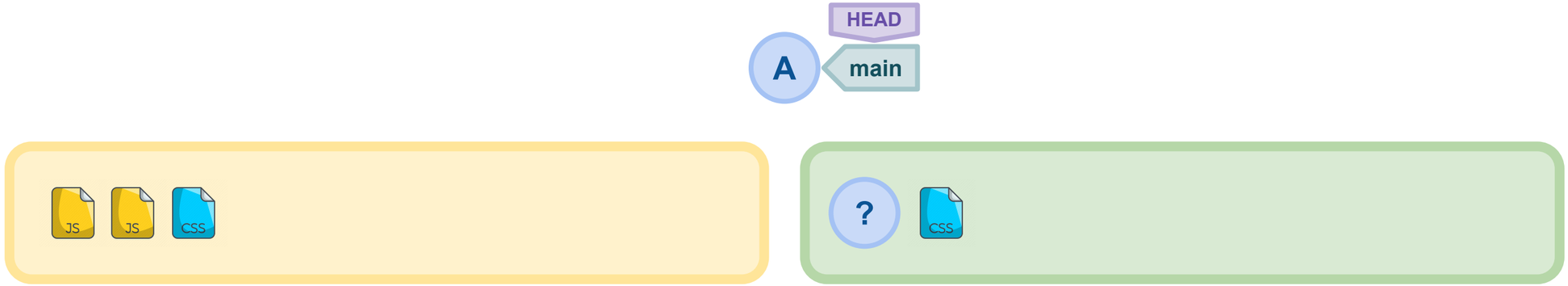
Git Simulator v.1.3.73 🧐

```
git add *.css
```



© All rights reserved. www.keepcoding.io

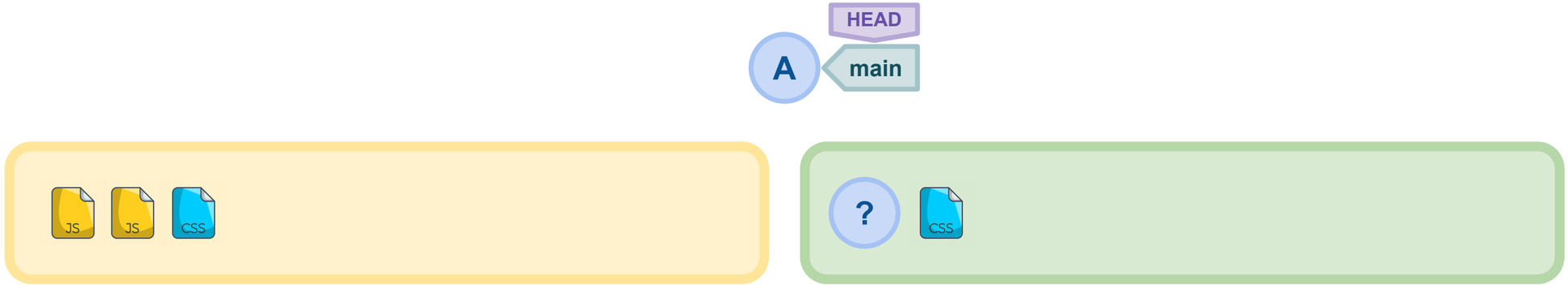
Git Simulator v.1.3.73 🧐



© All rights reserved. www.keepcoding.io

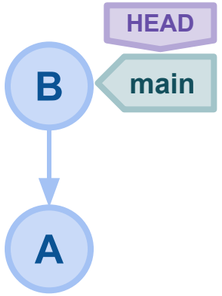
Git Simulator v.1.3.73 🧐

```
git commit -m “Mi segundo commit”
```



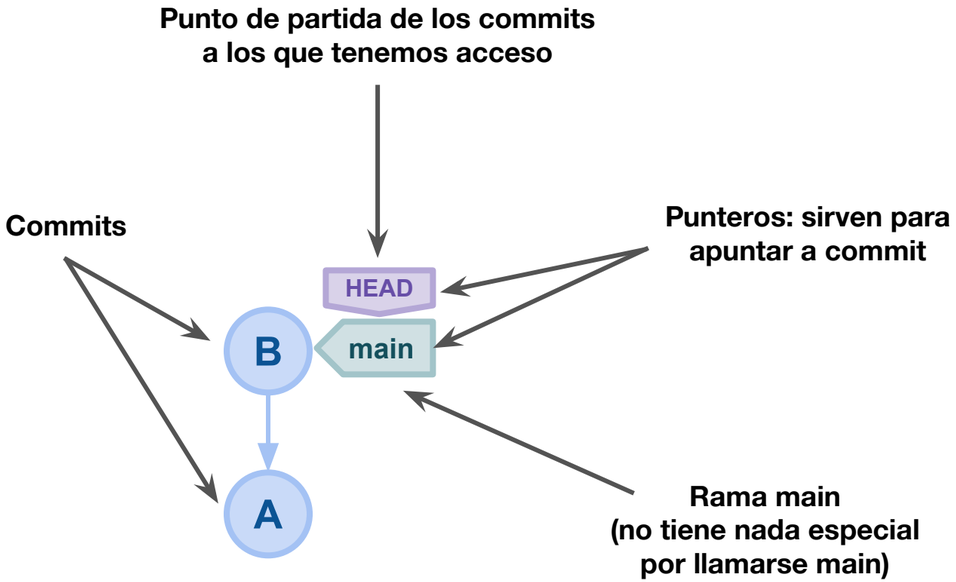
© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 🧐



© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 🧐



© All rights reserved. www.keepcoding.io

¿Cómo podemos ver eso desde la consola?

`git log`

```
# Muestra la info los commits a los que tenemos acceso desde HEAD
git log

# Muestra la info de los commits pintando un grafo
git log --graph
```

© All rights reserved. www.keepcoding.io

¿Cómo podemos el contenido de un commit?

git show

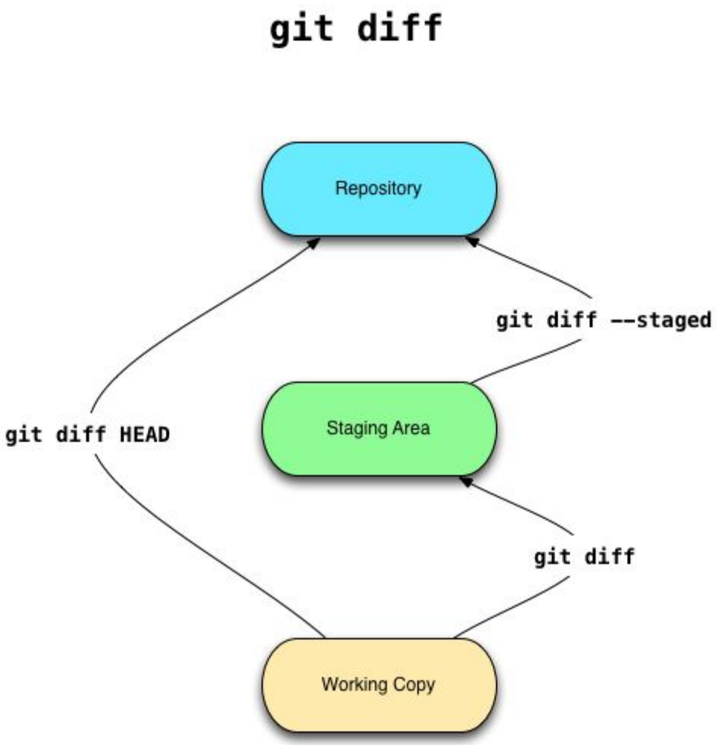
```
# Muestra el contenido del commit que indicamos
git show <commit id>

# Muestra el contenido del commit al que apunta HEAD
git show HEAD

# Muestra el contenido del commit al que apunta la rama main
git show main
```

© All rights reserved. www.keepcoding.io

¿Cómo podemos comparar las tres zonas?



© All rights reserved. www.keepcoding.io

Donde dije digo, digo Diego

Deshaciendo lo que hemos hecho

© All rights reserved. www.keepcoding.io

Deshaciendo cosas

Antes de deshacer nada…hay que pensar

¿Qué es lo que quieres deshacer?

© All rights reserved. www.keepcoding.io

¿Qué queremos deshacer?

- *“He picado código durante 2 horas y he tocado tantas cosas que quiero descartar todos los cambios”*
- *“Tras picar código durante 5 horas, he hecho el commit pero me he equivocado en el mensaje (insultaba al jefe).”*
- *“He picado código durante 3 horas, he hecho el commit pero sigue sin funcionar nada. Quiero volver a la versión del código anterior”.*

© All rights reserved. www.keepcoding.io

¿Qué queremos deshacer?

¿Qué queremos deshacer?	Comando	Zonas afectadas
Los cambios realizados	`git restore <file>`	Working Copy
El último commit	`git reset HEAD~1`	Graph (mentira)
El último commit + y los cambios realizados		Working Copy Graph (mentira)

© All rights reserved. www.keepcoding.io

¿Qué queremos deshacer?

¿Qué queremos deshacer?	Comando	Zonas afectadas
Los cambios realizados	`git restore <file>`	Working Copy
El último commit	`git reset HEAD~1`	Graph (mentira)

El último commit + y los cambios realizados	`git reset HEAD~1`
	+
	`git restore <file>`
		Working Copy
		Graph (mentira)

© All rights reserved. www.keepcoding.io

git restore

```
# Descarta los cambios realizados en el archivo <file>
git restore <file>

# Descarta los cambios realizados en archivos de la carpeta <file>
git restore <file>

# Descarta los cambios realizados en el archivos con extensión .md
git restore *.md
```

- Los cambios que hemos realizado en los archivos que le digamos.
- Sólo afecta a los archivos en **Working Copy**
- Realmente, lo que hace es volver a cargar la versión anterior del archivo
  - Por eso no podemos descartar archivos nuevos (que nunca han sido versionados)

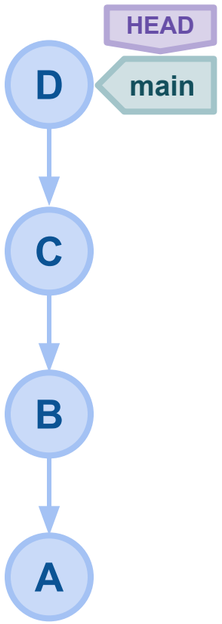
© All rights reserved. www.keepcoding.io

git reset

¿Qué pasa en el **Graph** cuando ejecutamos `git reset HEAD~1`?

© All rights reserved. www.keepcoding.io

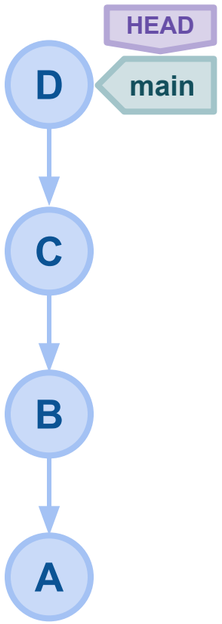
Git Simulator v.1.3.73 🤩



© All rights reserved. www.keepcoding.io

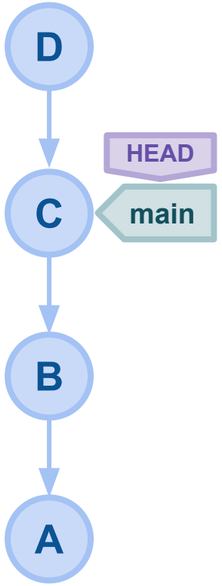
Git Simulator v.1.3.73 🤩

```
git reset HEAD~1
```



© All rights reserved. www.keepcoding.io

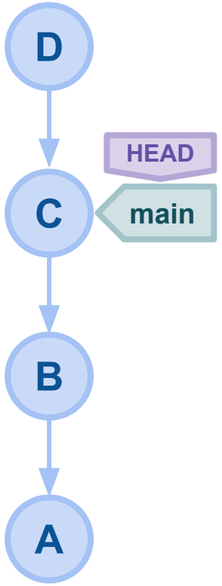
Git Simulator v.1.3.73 🤩



© All rights reserved. www.keepcoding.io

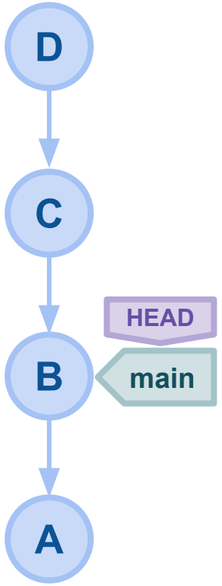
Git Simulator v.1.3.73 🤩

```
git reset HEAD~1
```



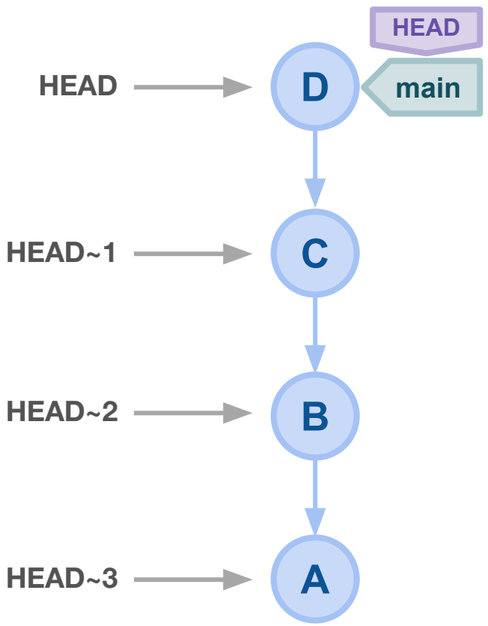
© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 🤩



© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 🤩



© All rights reserved. www.keepcoding.io

`git reset`

Ok, entonces

¿para qué sirve el comando `git reset`?

© All rights reserved. www.keepcoding.io

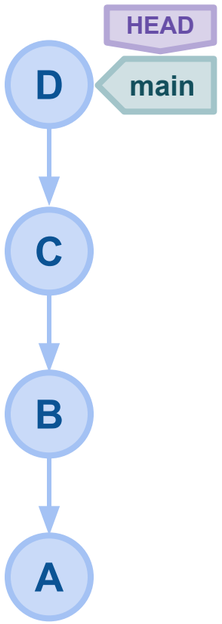
git reset

¡MENTIRA!

~~Para deshacer un commit~~

© All rights reserved. www.keepcoding.io

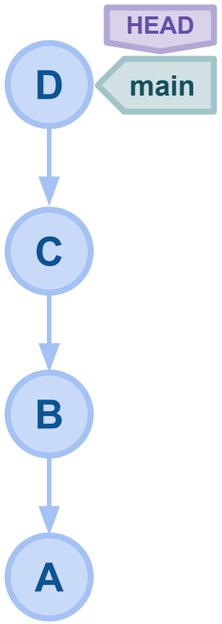
Git Simulator v.1.3.73 🤠



© All rights reserved. www.keepcoding.io

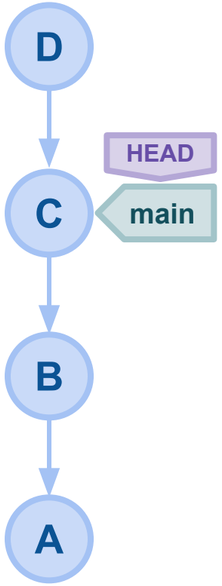
Git Simulator v.1.3.73 🤠

```
git reset HEAD~1
```



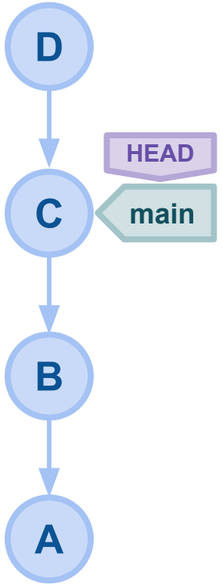
© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 🤠



© All rights reserved. www.keepcoding.io

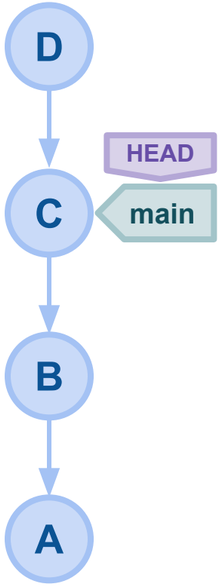
Git Simulator v.1.3.73 🤓



© All rights reserved. www.keepcoding.io

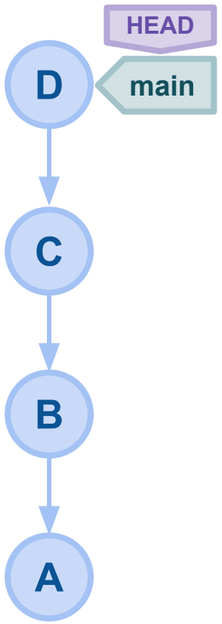
Git Simulator v.1.3.73 🤓

```
git reset D
```



© All rights reserved. www.keepcoding.io

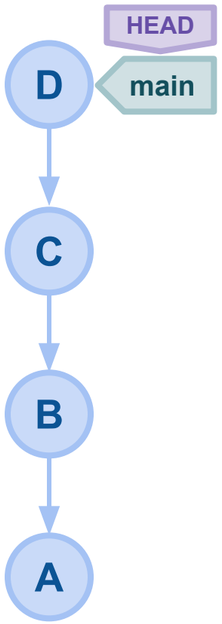
Git Simulator v.1.3.73 🤓



© All rights reserved. www.keepcoding.io

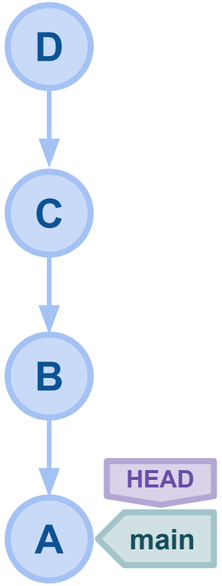
Git Simulator v.1.3.73 🤓

```
git reset A
```



© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 🤓



© All rights reserved. www.keepcoding.io

`git reset`

Sirve para mover una rama
allá donde nosotros queramos.

`git reset` mueve ramas

© All rights reserved. www.keepcoding.io

¿Qué queremos deshacer?

¿Qué queremos deshacer?	Comando	Zonas afectadas
Los cambios realizados	`git restore <file>`	Working Copy
El último commit	`git reset HEAD~1`	Graph (mentira)

El último commit + y los cambios realizados	`git reset HEAD~1`
	+
	`git restore <file>`
		Working Copy
		Graph (mentira)

© All rights reserved. www.keepcoding.io

¿Qué queremos deshacer?

¿Qué queremos deshacer?	Comando	Zonas afectadas
Los cambios realizados	`git restore <file>`	Working Copy
El último commit	`git reset HEAD~1`	Graph (mentira)
El último commit + y los cambios realizados	`git reset --hard HEAD~1`	Working Copy Graph (mentira)

© All rights reserved. www.keepcoding.io

¿Y cómo deshago lo que pongo en **Staging Area**?

```
❯ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   readme.md
```

```
git restore --staged <file>
```

© All rights reserved. www.keepcoding.io

`git clean`

Elimina cualquier archivo sin trackear del repositorio.

Muy útil para quitar archivos generados o que no valen.

- `git clean -n` - dry run, muestra lo que sería eliminado
- `git clean -f` - borrado forzoso
- `git clean -fd` - elimina archivos y directorios
- `git clean -i` - modo interactivo

© All rights reserved. www.keepcoding.io

¿Qué pasa si borramos un archivo?

- Para Git, crear un archivo de 10 líneas es: un cambio en un archivo que pasa de tener 0 líneas a tener 10
- Borrar un archivo de 500 líneas es: ????????????

© All rights reserved. www.keepcoding.io

¿Qué pasa si borramos un archivo?

- Para Git, crear un archivo de 10 líneas es: un cambio en un archivo que pasa de tener 0 líneas a tener 10
- Borrar un archivo de 500 líneas es: un cambio en un archivo que pasa de tener 500 líneas a tener 0

© All rights reserved. www.keepcoding.io

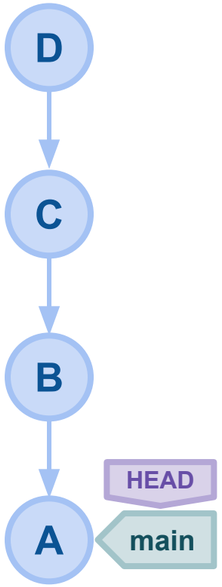
¿Qué pasa si borramos un archivo?

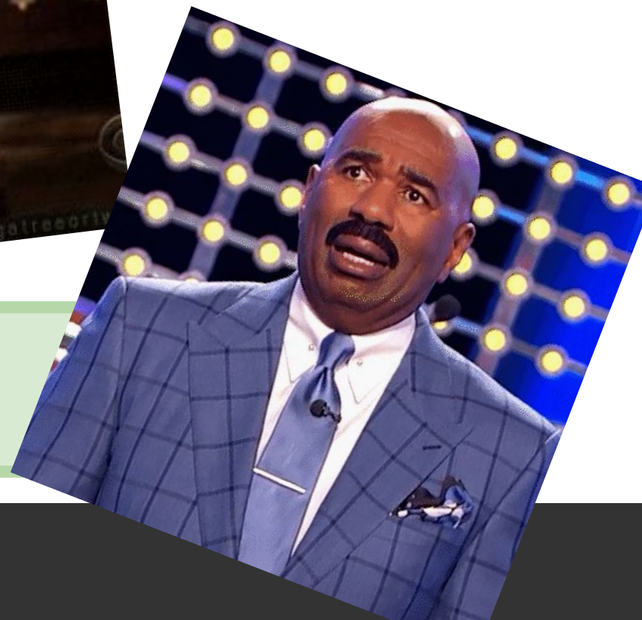
- Para Git, crear un archivo de 10 líneas es: un cambio en un archivo que pasa de tener 0 líneas a tener 10
- Borrar un archivo de 500 líneas es: un cambio en un archivo que pasa de tener 500 líneas a tener 0
- A Git sólo le importan los cambios dentro de los archivos, no los archivos en sí

© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 🤓

¿Y ahora cómo volvemos a D?





© All rights reserved. www.keepcoding.io

`git reflog` al rescate

Cuando las cosas se ponen feas…

```
git reflog
```

- Es un log que registra todos los commits por los que pasa HEAD
- Además, nos dice con qué comando se llegó a ese commit
- Es como un rastro de migas

© All rights reserved. www.keepcoding.io

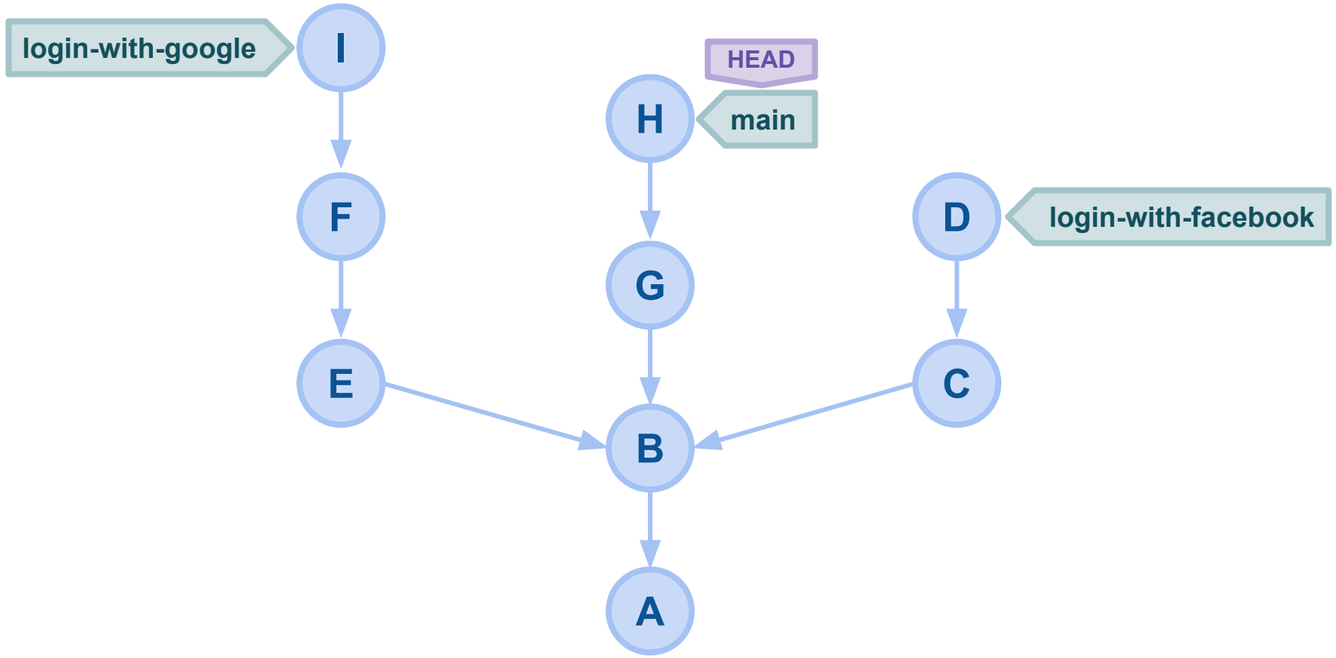
Ramas

Branches para los amigos

© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 😎

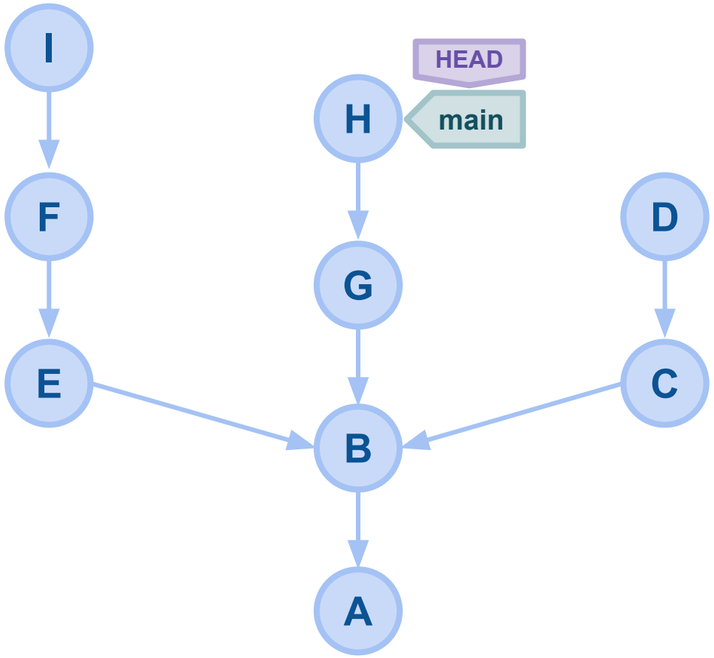
¿Cuántas ramas tiene este repo?



© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 😎

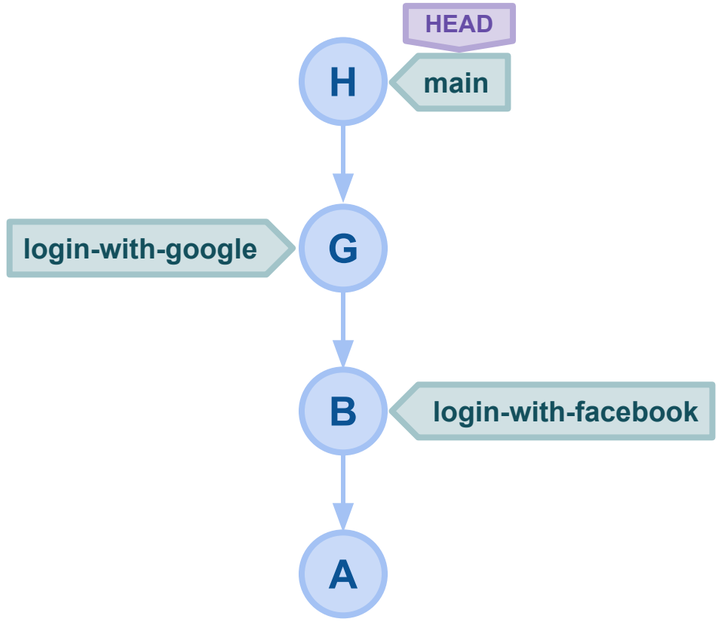
¿Y éste?



© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 😎

¿Y éste?



© All rights reserved. www.keepcoding.io

Ver las ramas existentes

```
git branch
```

- Muestra las ramas existentes en el repo
- Marca con * la rama en está HEAD

© All rights reserved. www.keepcoding.io

Crear una rama

git branch

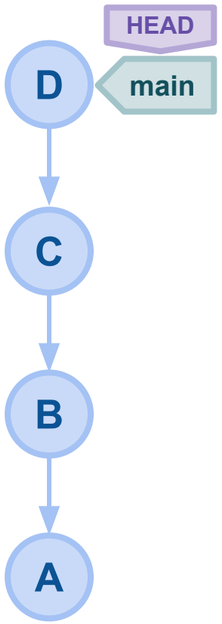
```
# Crea la rama <branch name> en el commit al que apunta HEAD
git branch <branch name>

# Crea la rama <branch name> en el commit <commit id>
git branch <branch name> <commit id>

# Crea la rama <branch name> en el commit al que apunta la rama main
git branch <branch name> main
```

© All rights reserved. www.keepcoding.io

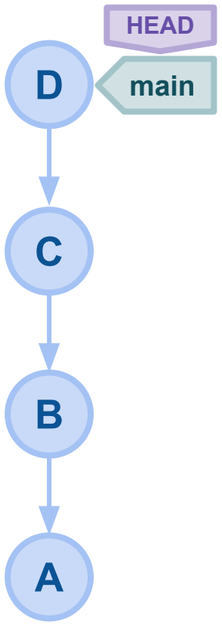
Git Simulator v.1.3.73 😘



© All rights reserved. www.keepcoding.io

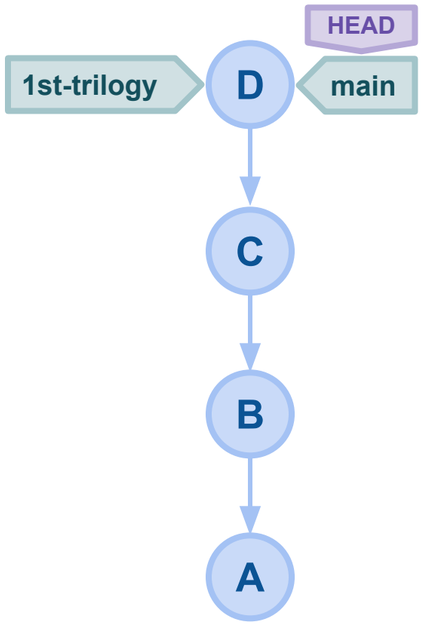
Git Simulator v.1.3.73 😘

```
git branch 1st-trilogy
```



© All rights reserved. www.keepcoding.io

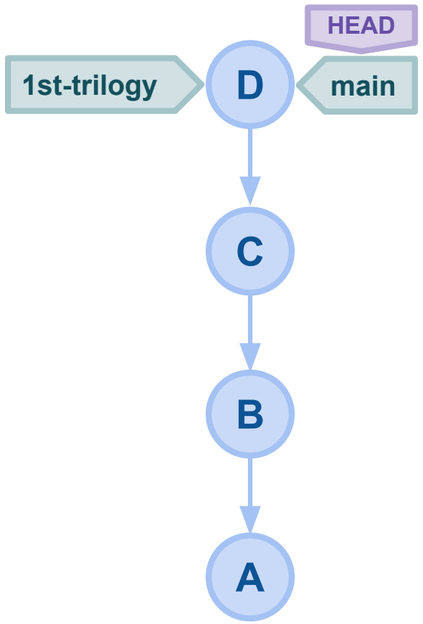
Git Simulator v.1.3.73 😘



© All rights reserved. www.keepcoding.io

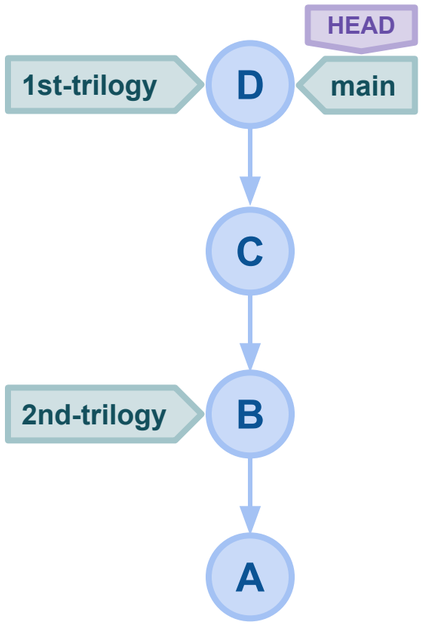
Git Simulator v.1.3.73 😘

```
git branch 2nd-trilogy B
```



© All rights reserved. www.keepcoding.io

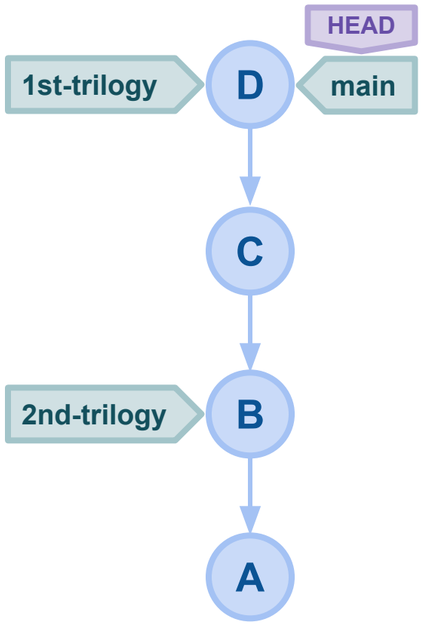
Git Simulator v.1.3.73 😘



© All rights reserved. www.keepcoding.io

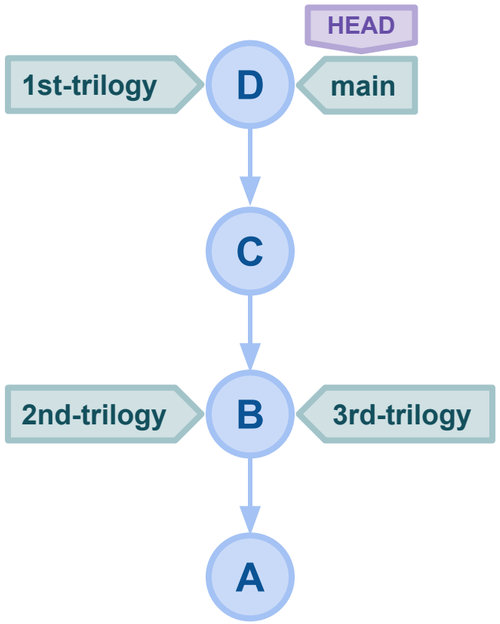
Git Simulator v.1.3.73 😘

```
git branch 3rd-trilogy 2nd-trilogy
```



© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 😘



© All rights reserved. www.keepcoding.io

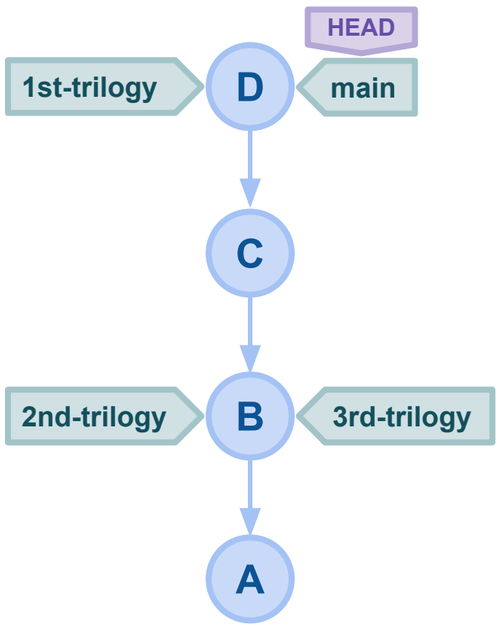
Cambiarse de una rama a otra

```
git checkout
```

- Permite cambiar de una rama a otra
- ⚠️ Al cambiar de rama, cambia el contenido del **Working Copy** ⚠️

© All rights reserved. www.keepcoding.io

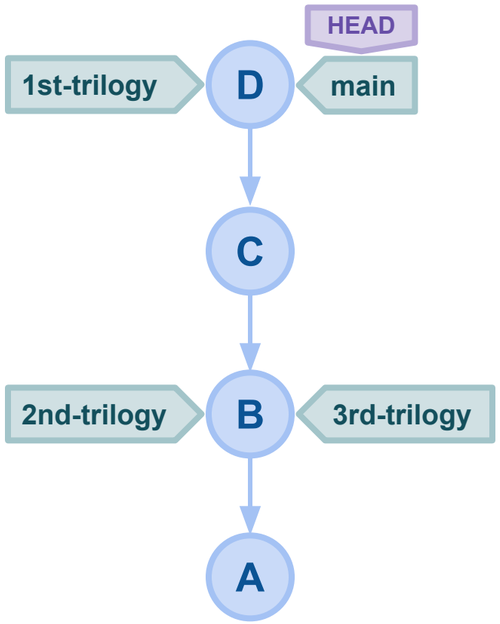
Git Simulator v.1.3.73 🤯



© All rights reserved. www.keepcoding.io

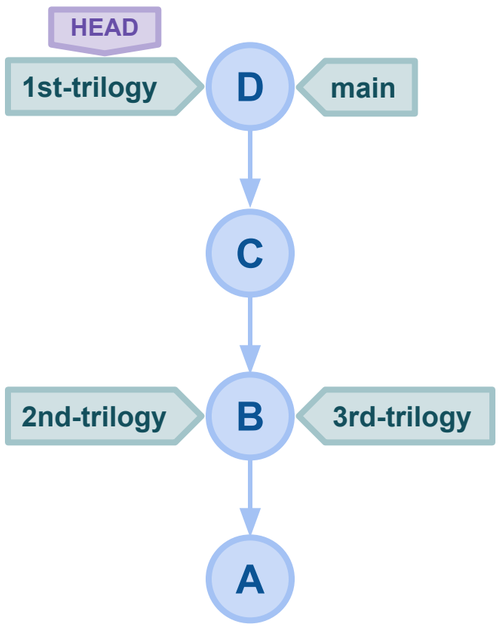
Git Simulator v.1.3.73 🤯

```
git checkout 1st-trilogy
```



© All rights reserved. www.keepcoding.io

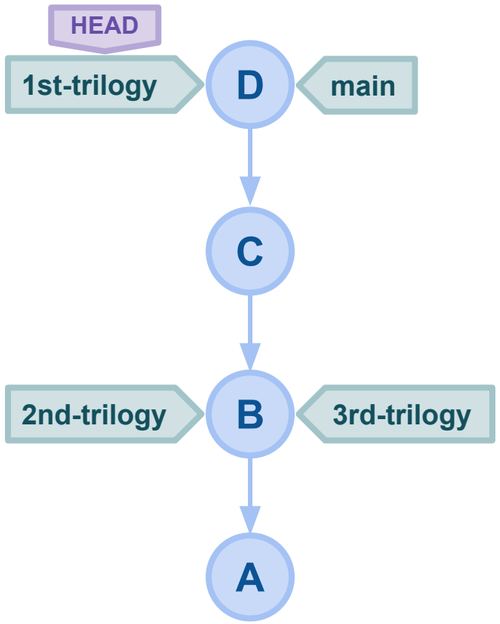
Git Simulator v.1.3.73 🤯



© All rights reserved. www.keepcoding.io

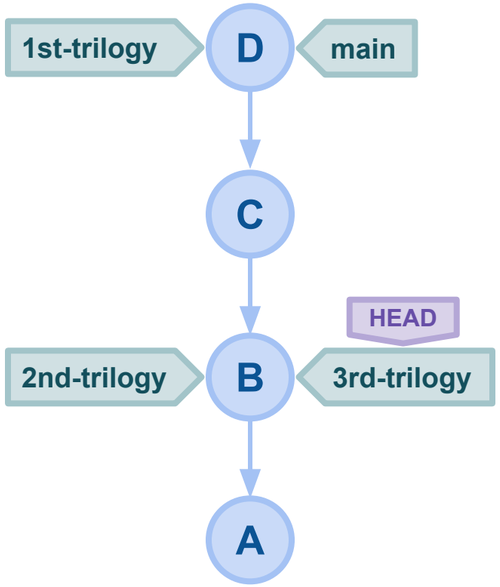
Git Simulator v.1.3.73 🤯

```
git checkout 3rd-trilogy
```



© All rights reserved. www.keepcoding.io

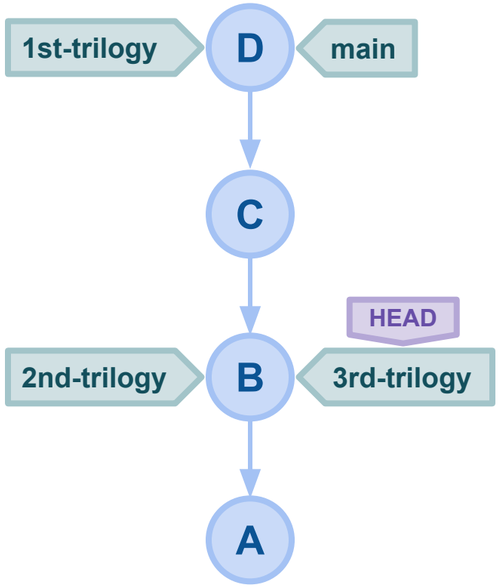
Git Simulator v.1.3.73 🤯



© All rights reserved. www.keepcoding.io

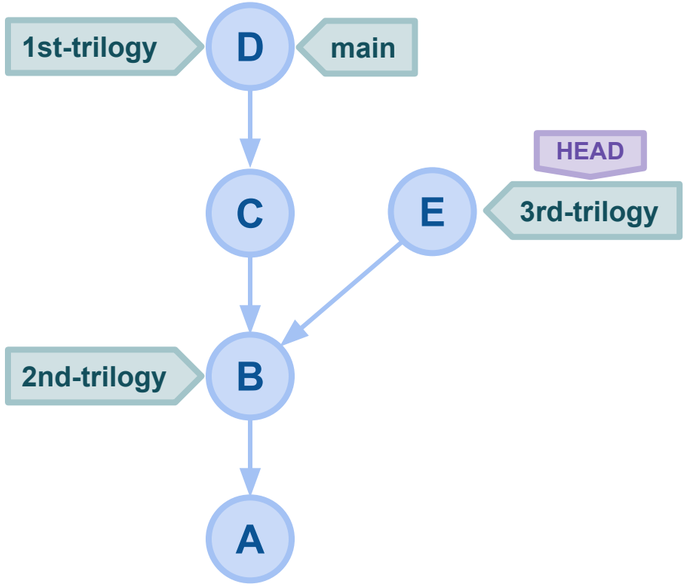
Git Simulator v.1.3.73 🤯

```
git commit -m “E”
```



© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 🤯



© All rights reserved. www.keepcoding.io

Eliminar una rama

`git branch`

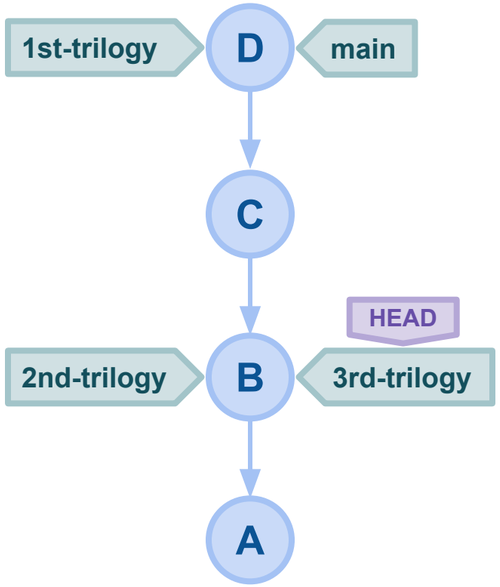
```
# Elimina la rama <branch name> siempre y cuando no deje commits inaccesibles
git branch -d <branch name>

# Elimina la rama <branch name> aunque deje commits inaccesibles
git branch -D <branch name>
```

⚠️ No podemos borrar la rama en la que estamos ⚠️

© All rights reserved. www.keepcoding.io

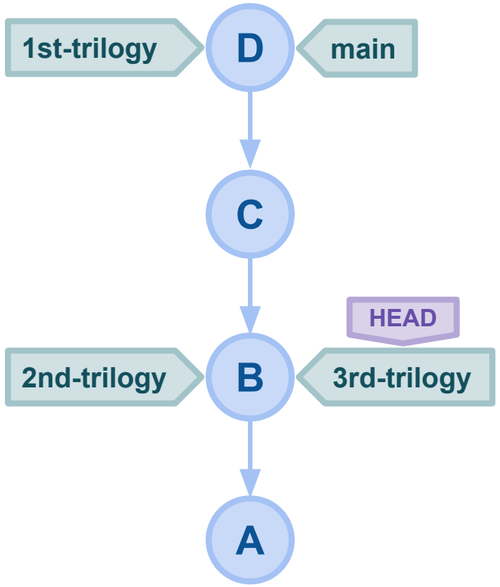
Git Simulator v.1.3.73 🤯



© All rights reserved. www.keepcoding.io

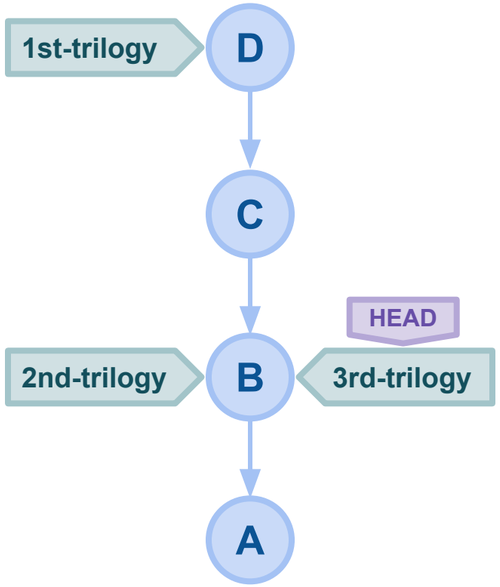
Git Simulator v.1.3.73 🤯

```
git branch -d main
```



© All rights reserved. www.keepcoding.io

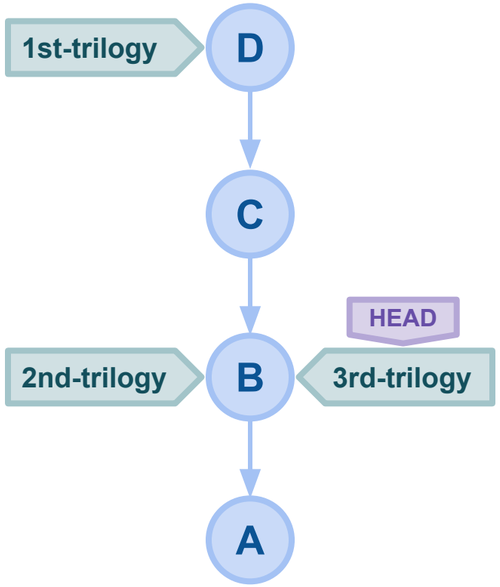
Git Simulator v.1.3.73 🤯



© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 🤯

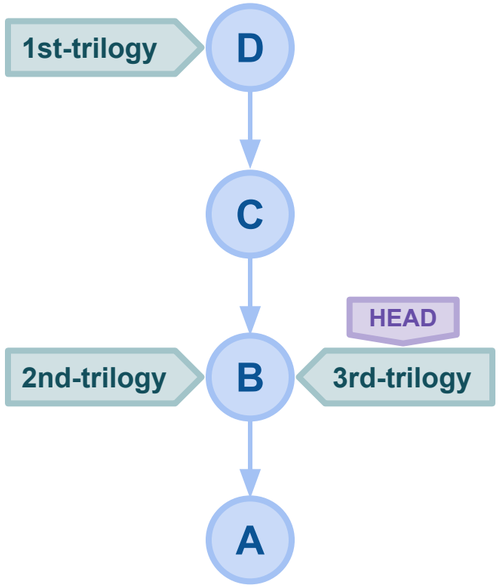
```
git branch -d 1st-trilogy
```



© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 🤯

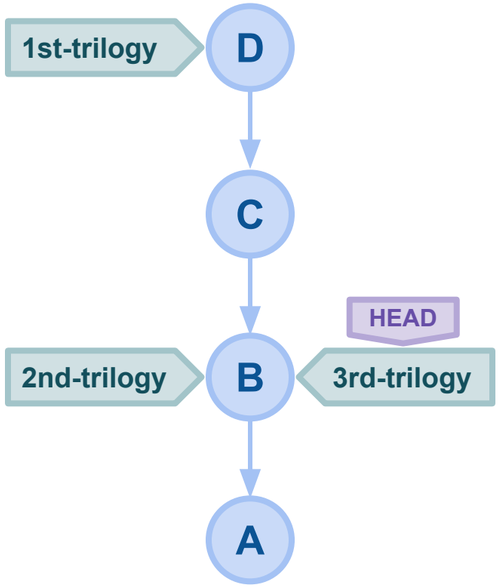
⚠️ ¡Error! ¿Estás seguro? ¡Que dejas commits inaccesibles! ⚠️



© All rights reserved. www.keepcoding.io

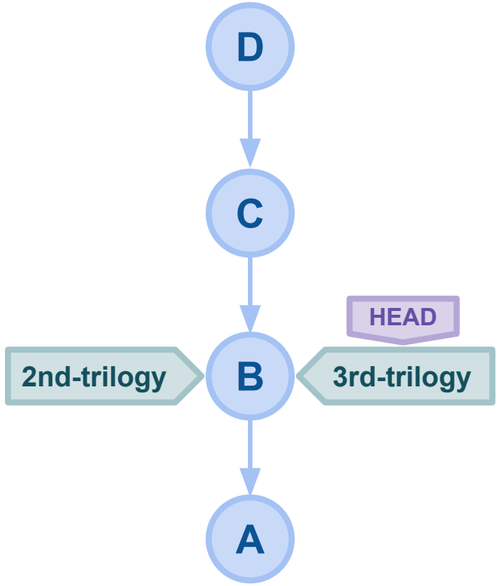
Git Simulator v.1.3.73 🤯

```
git branch -D 1st-trilogy
```



© All rights reserved. www.keepcoding.io

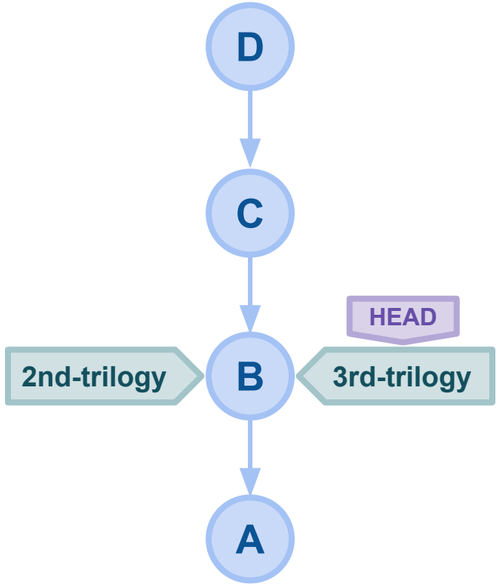
Git Simulator v.1.3.73 🤯



© All rights reserved. www.keepcoding.io

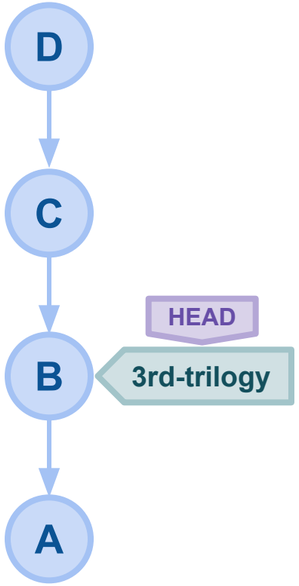
Git Simulator v.1.3.73 🤯

```
git branch -d 2nd-trilogy
```



© All rights reserved. www.keepcoding.io

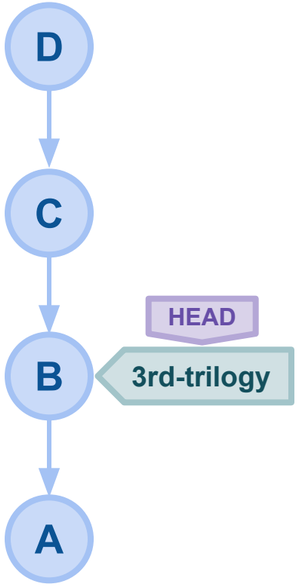
Git Simulator v.1.3.73 🤯



© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 🤯

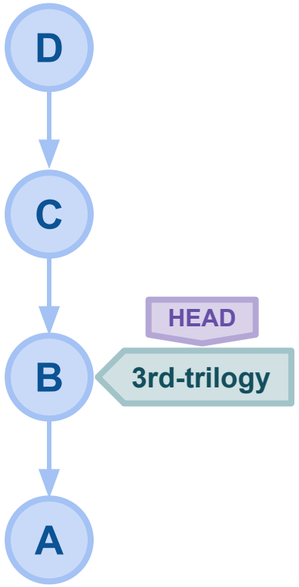
```
git branch -D 3rd-trilogy
```



© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 🤯

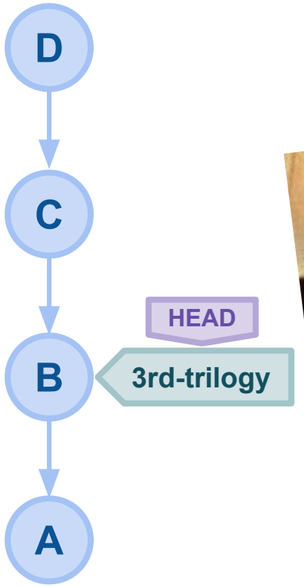
⚠️ ¡Error! No puedes eliminar la rama en la que estás ⚠️

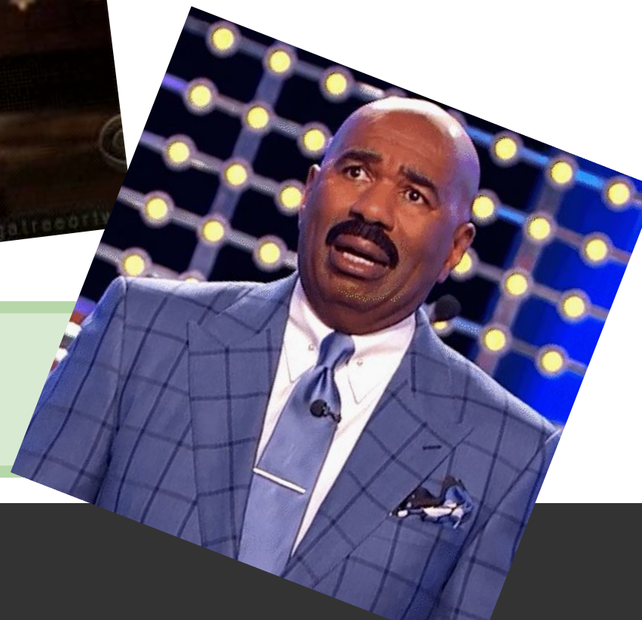


© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 🤯

¿Y ahora cómo recuperamos las ramas?





© All rights reserved. www.keepcoding.io

`git reflog` al rescate

Tags

Etiquetas para los amigos…y primas hermanas de las ramas

© All rights reserved. www.keepcoding.io

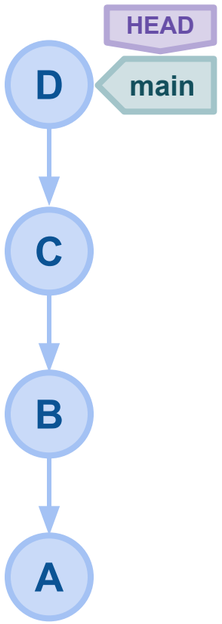
Tags

```
git tag
```

- Son como las ramas…pero que no se pueden mover
- Sirven para marcar hitos importantes en nuestro desarrollo:
  - Números de versión
  - Actualizaciones de framework

© All rights reserved. www.keepcoding.io

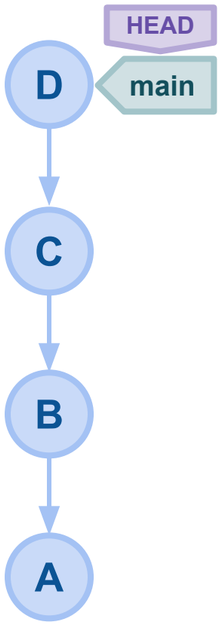
Git Simulator v.1.3.73 😅



© All rights reserved. www.keepcoding.io

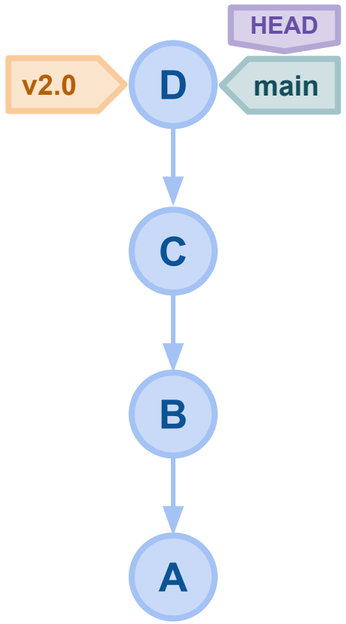
Git Simulator v.1.3.73 😅

```
git tag v2.0
```



© All rights reserved. www.keepcoding.io

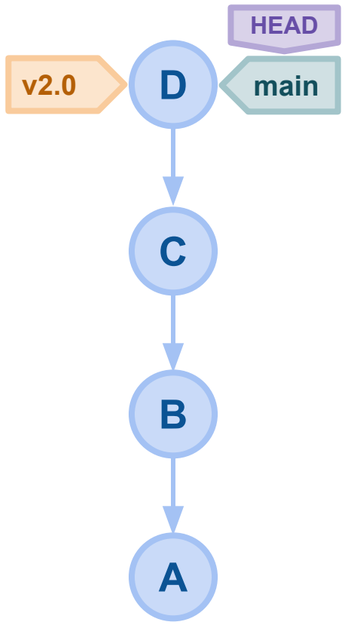
Git Simulator v.1.3.73 😅



© All rights reserved. www.keepcoding.io

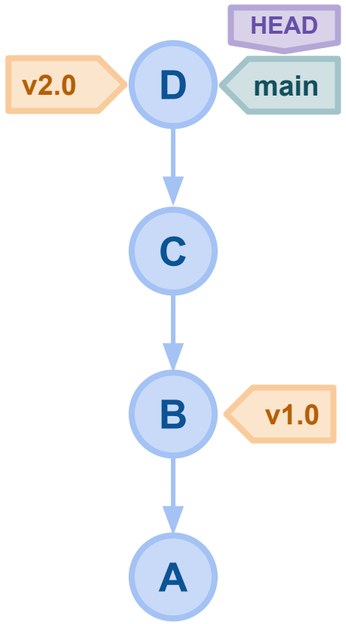
Git Simulator v.1.3.73 😅

```
git tag v1.0 B
```



© All rights reserved. www.keepcoding.io

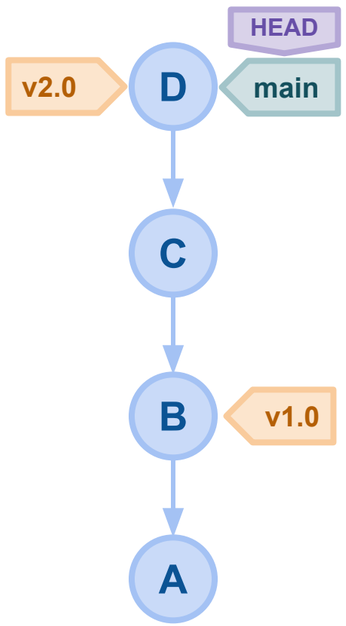
Git Simulator v.1.3.73 😅



© All rights reserved. www.keepcoding.io

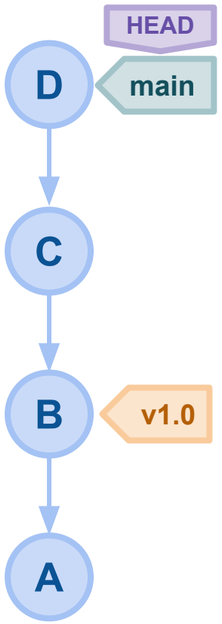
Git Simulator v.1.3.73 😅

```
git tag -d v2.0
```



© All rights reserved. www.keepcoding.io

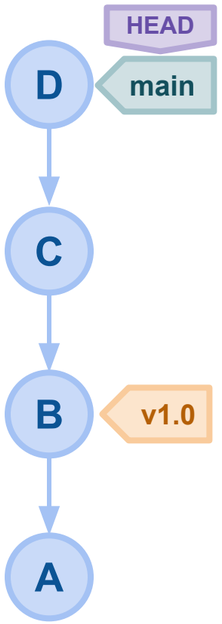
Git Simulator v.1.3.73 😅



© All rights reserved. www.keepcoding.io

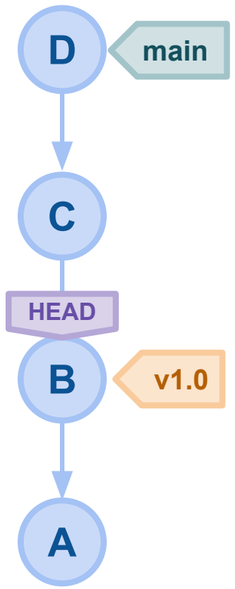
Git Simulator v.1.3.73 😅

```
git checkout v1.0
```



© All rights reserved. www.keepcoding.io

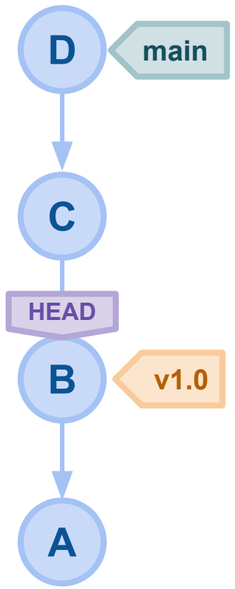
Git Simulator v.1.3.73 😅



© All rights reserved. www.keepcoding.io

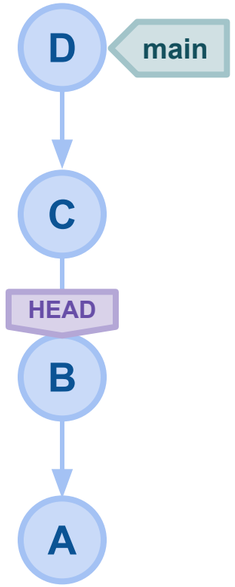
Git Simulator v.1.3.73 😅

```
git tag -d v1.0
```



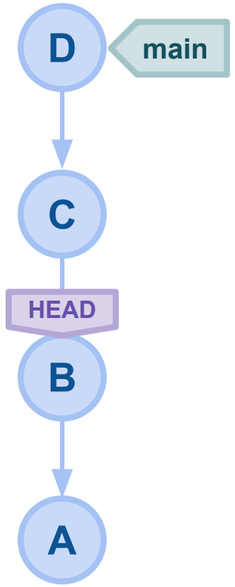
© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 😅



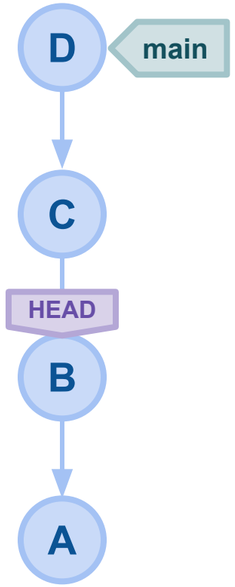
© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 😅



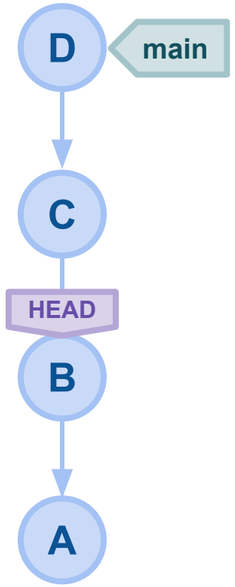
© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 😅



© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 😅



© All rights reserved. www.keepcoding.io

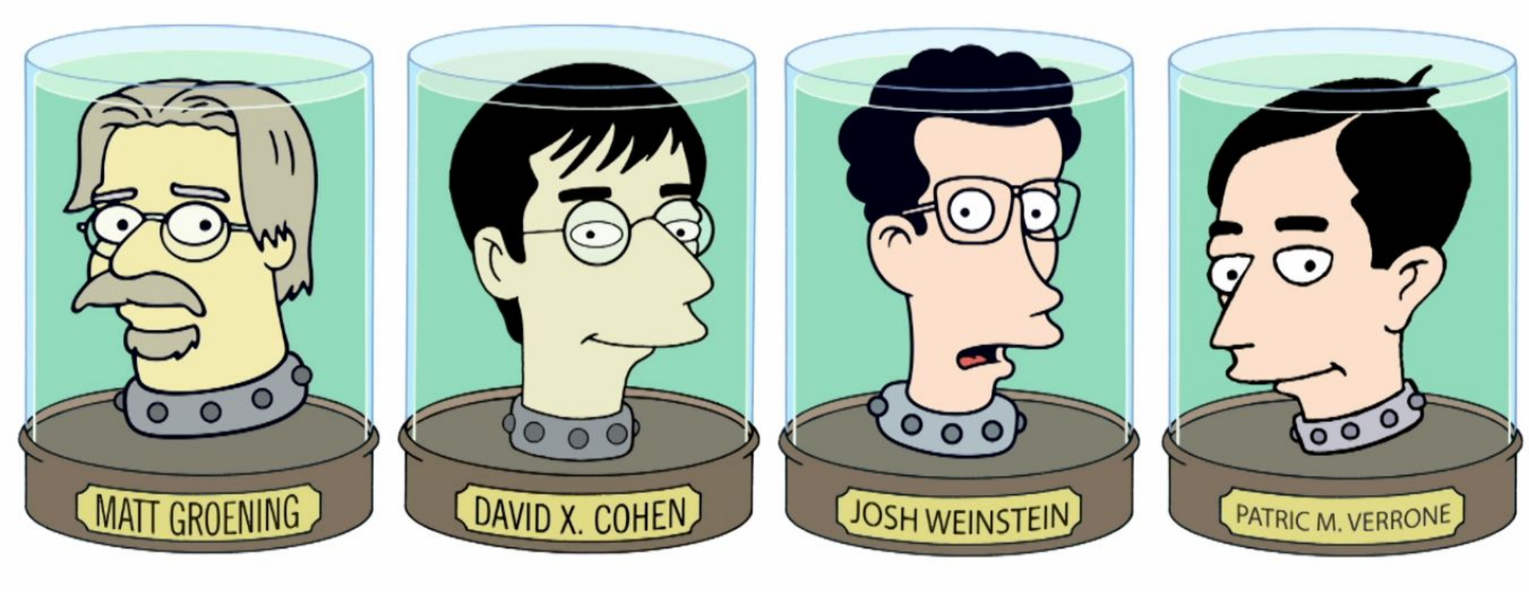
git checkout

¿Y qué pasa cuando no estamos en una rama?

© All rights reserved. www.keepcoding.io

git checkout

Que estaremos en un estado *DETACHED HEAD*



© All rights reserved. www.keepcoding.io

`git checkout`

Ok, entonces

¿para qué sirve el comando `git checkout`?

© All rights reserved. www.keepcoding.io

git checkout

¡MENTIRA!

~~Para cambiar de una rama a otra~~

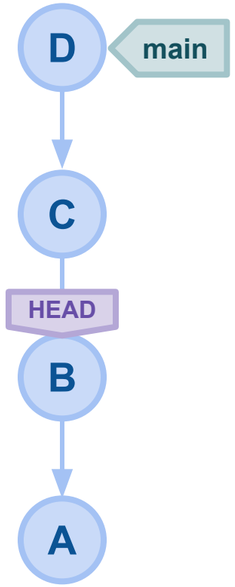
© All rights reserved. www.keepcoding.io

git checkout

Sirve para mover HEAD
allá donde nosotros queramos.

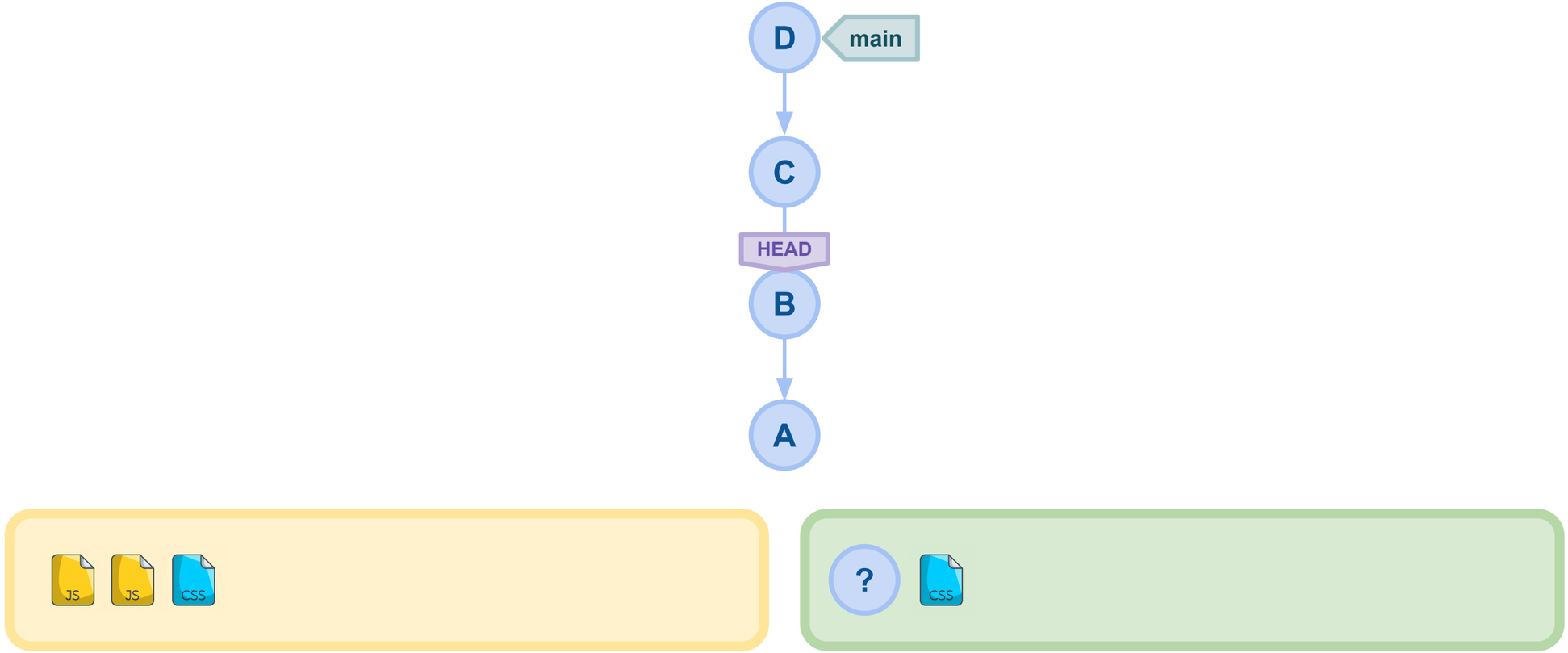
© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 😵



© All rights reserved. www.keepcoding.io

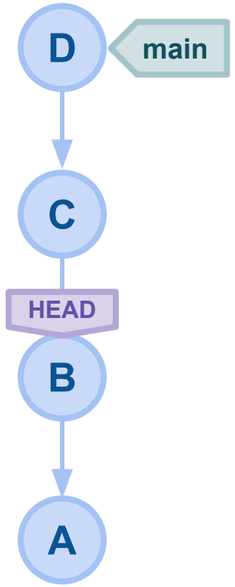
Git Simulator v.1.3.73 😵

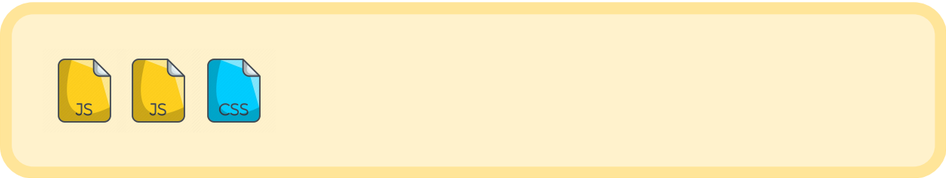


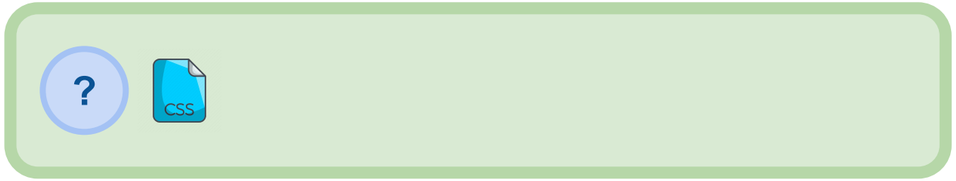
© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 😵

```
git commit -m “Commit experimental”
```

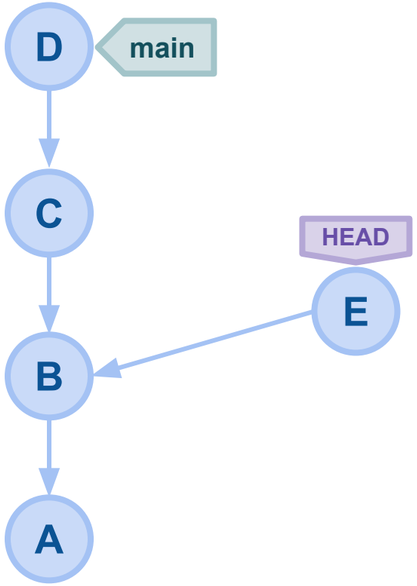






© All rights reserved. www.keepcoding.io

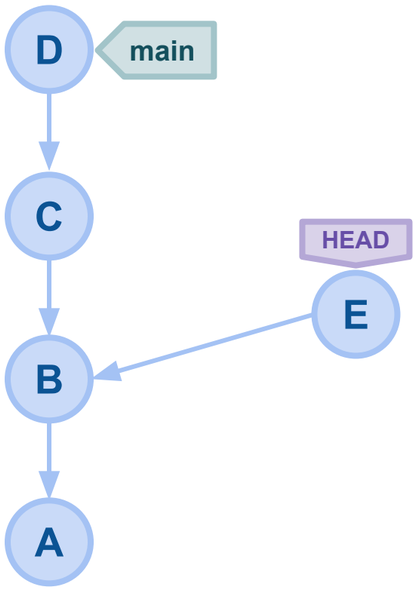
Git Simulator v.1.3.73 😵



© All rights reserved. www.keepcoding.io

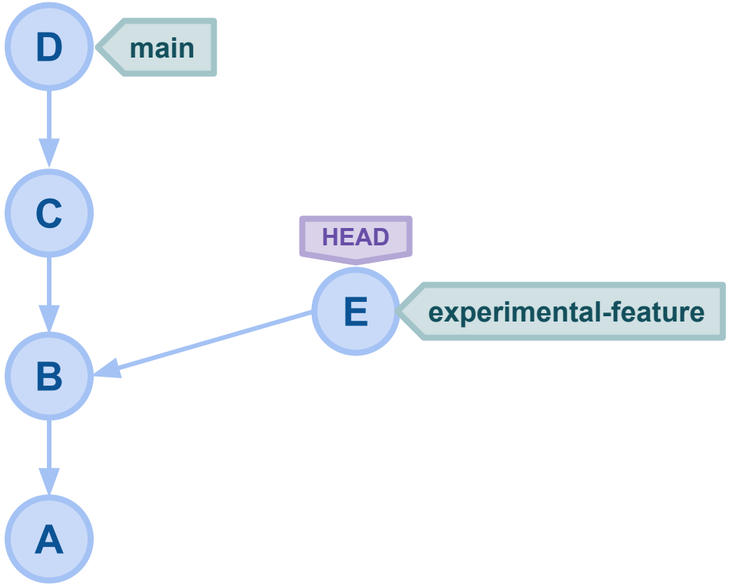
Git Simulator v.1.3.73 😵

```
git branch experimental-feature
```



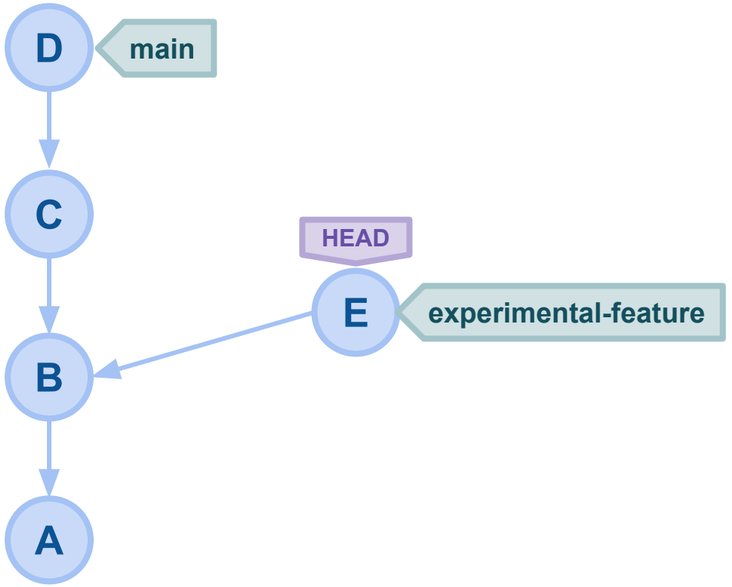
© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 😵



© All rights reserved. www.keepcoding.io

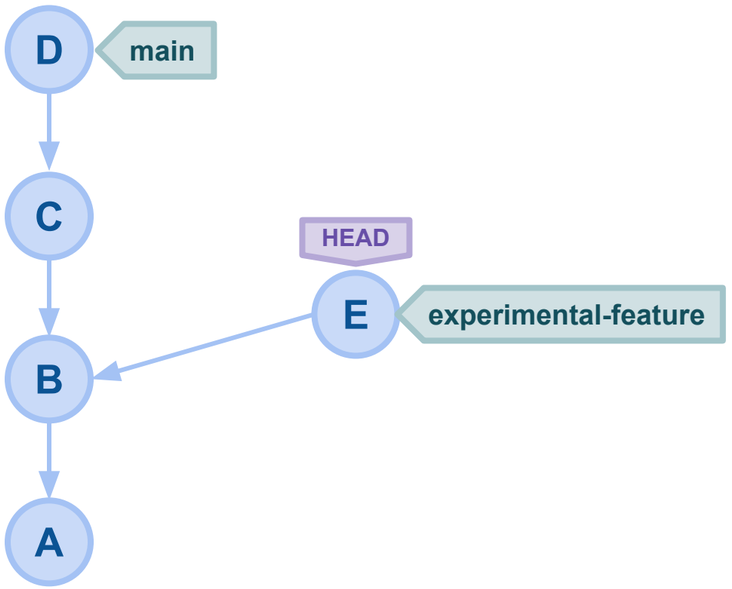
Git Simulator v.1.3.73 😵

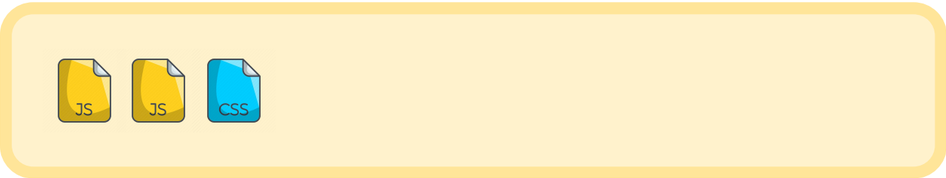


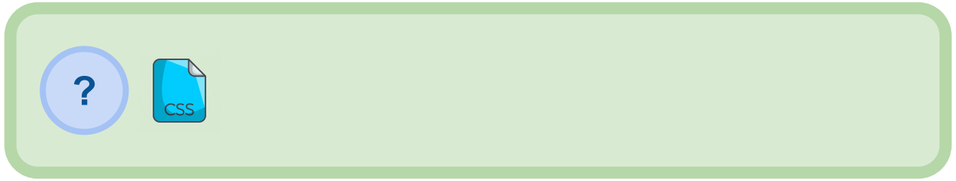
© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 😵

```
git commit -m “Otra prueba experimental”
```

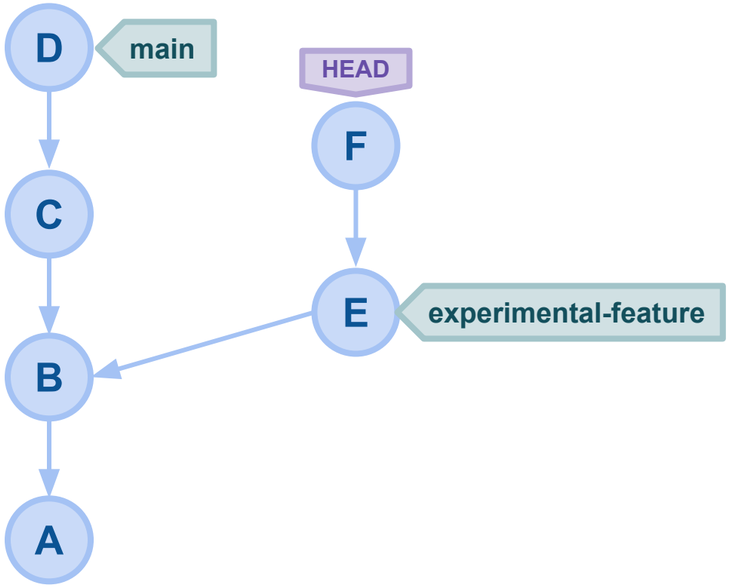






© All rights reserved. www.keepcoding.io

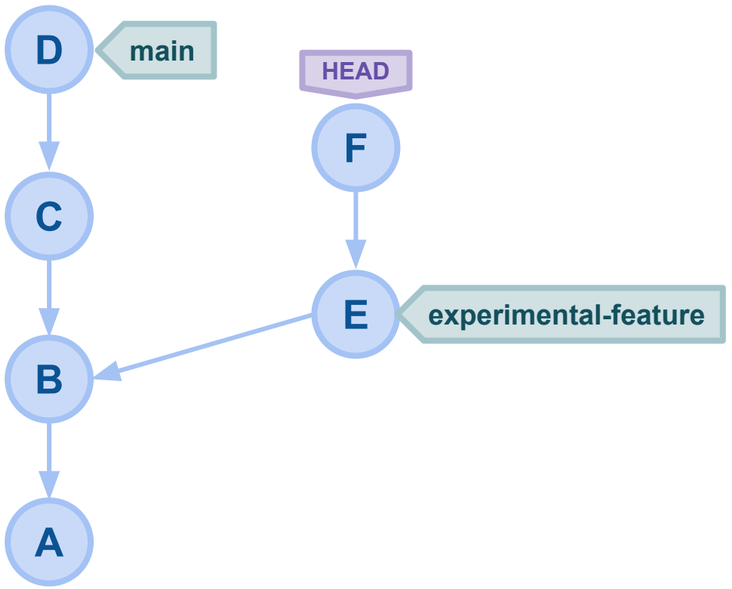
Git Simulator v.1.3.73 😵



© All rights reserved. www.keepcoding.io

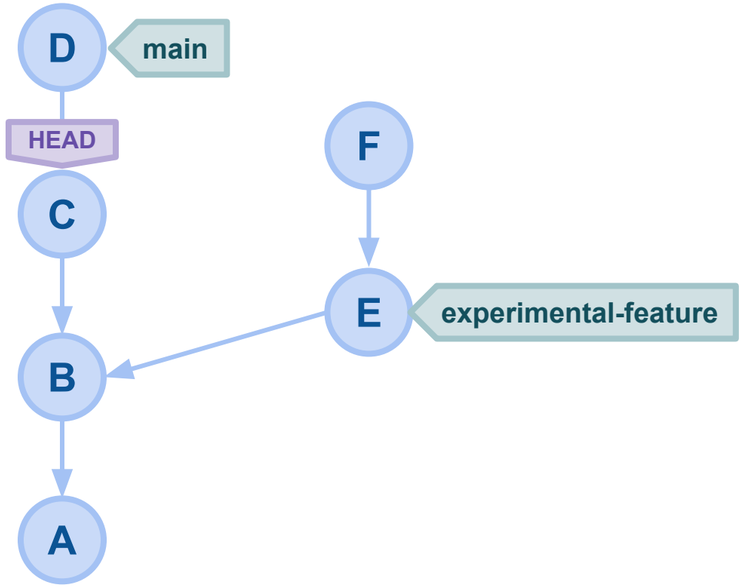
Git Simulator v.1.3.73 😵

```
git checkout C
```



© All rights reserved. www.keepcoding.io

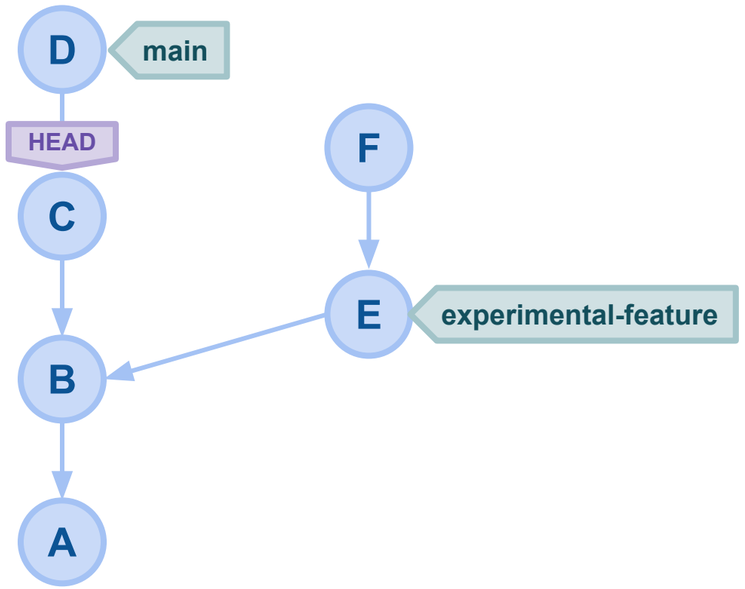
Git Simulator v.1.3.73 😵



© All rights reserved. www.keepcoding.io

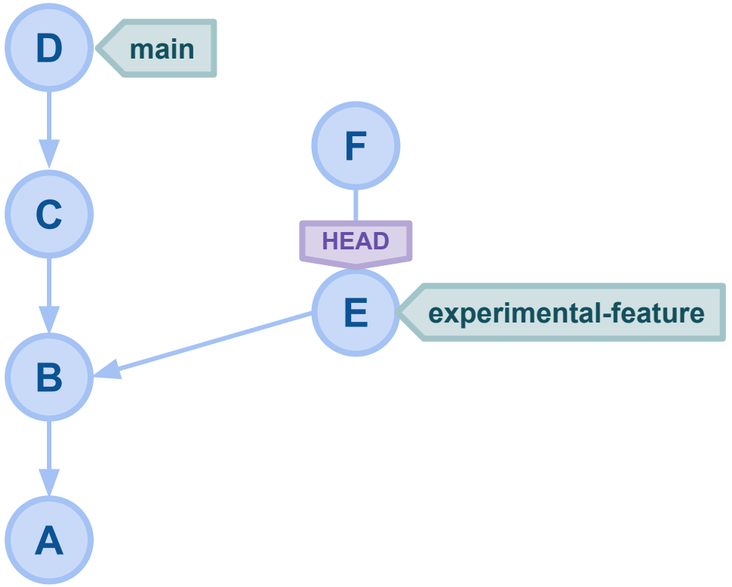
Git Simulator v.1.3.73 😵

```
git checkout E
```



© All rights reserved. www.keepcoding.io

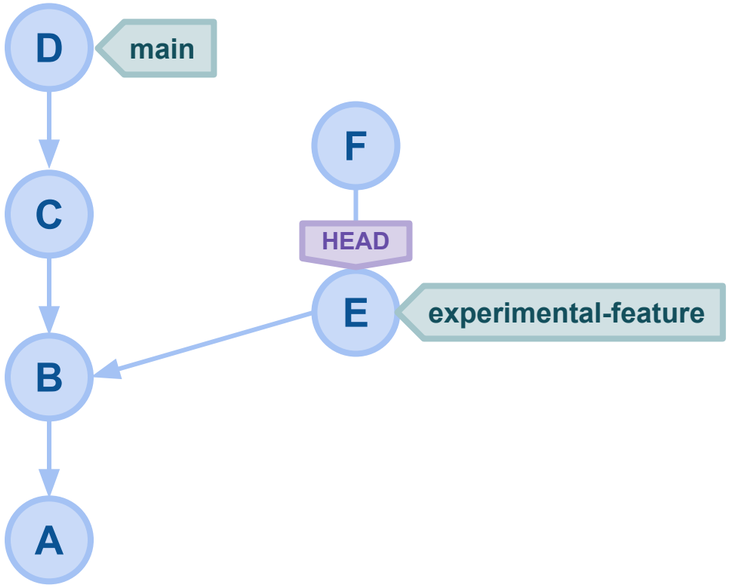
Git Simulator v.1.3.73 😵



© All rights reserved. www.keepcoding.io

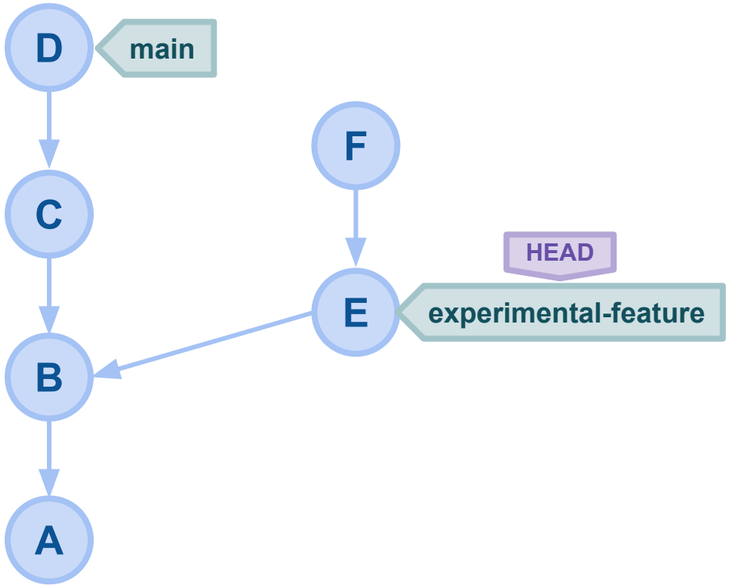
Git Simulator v.1.3.73 😵

```
git checkout experimental-feature
```



© All rights reserved. www.keepcoding.io

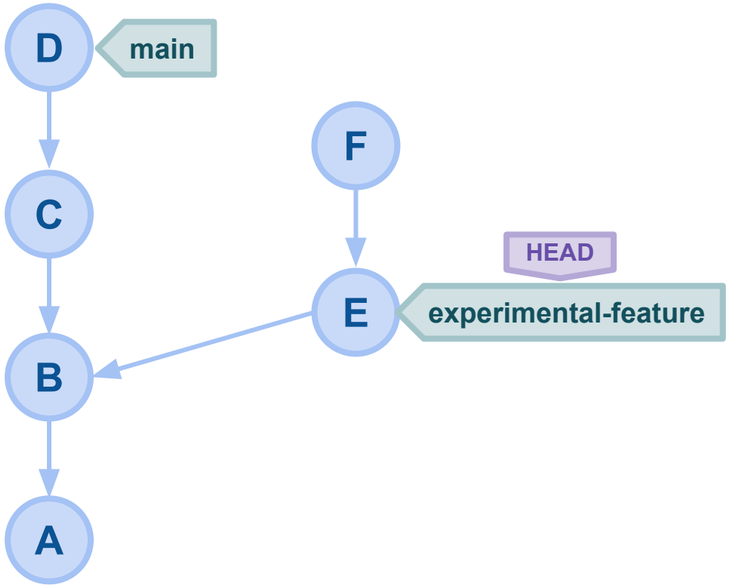
Git Simulator v.1.3.73 😵



© All rights reserved. www.keepcoding.io

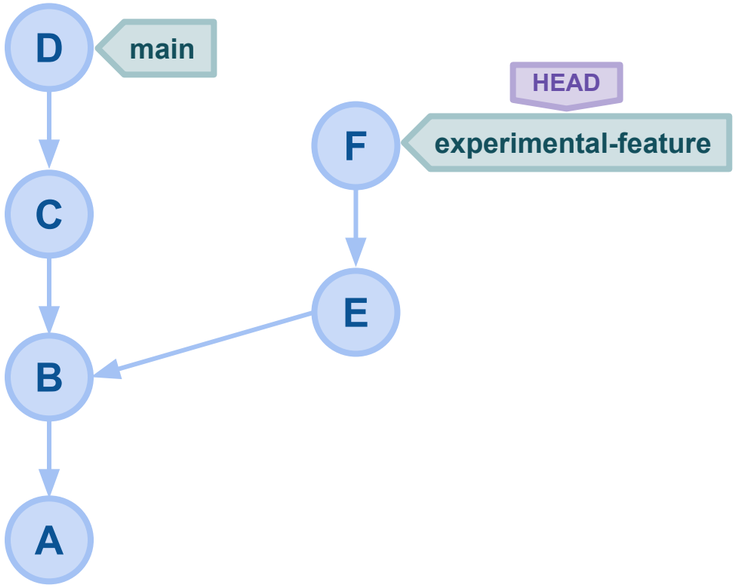
Git Simulator v.1.3.73 😵

```
git reset --hard F
```



© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 😵



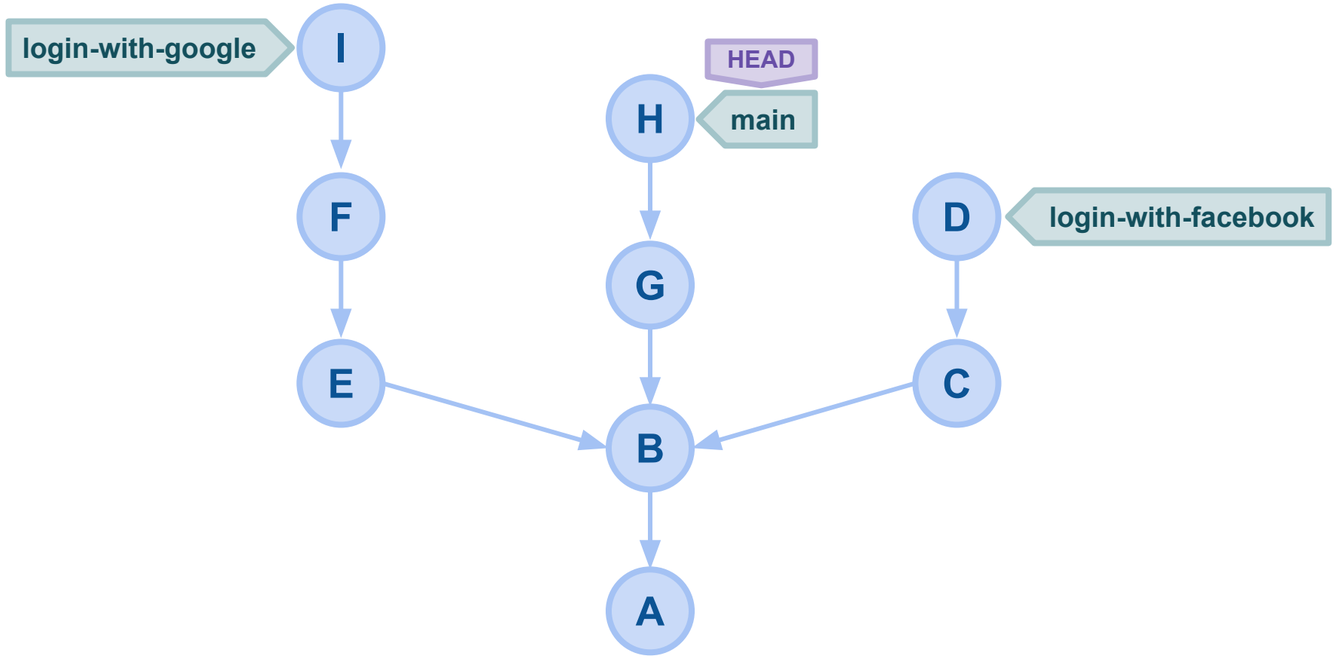
© All rights reserved. www.keepcoding.io

Merging

Fusionando branches

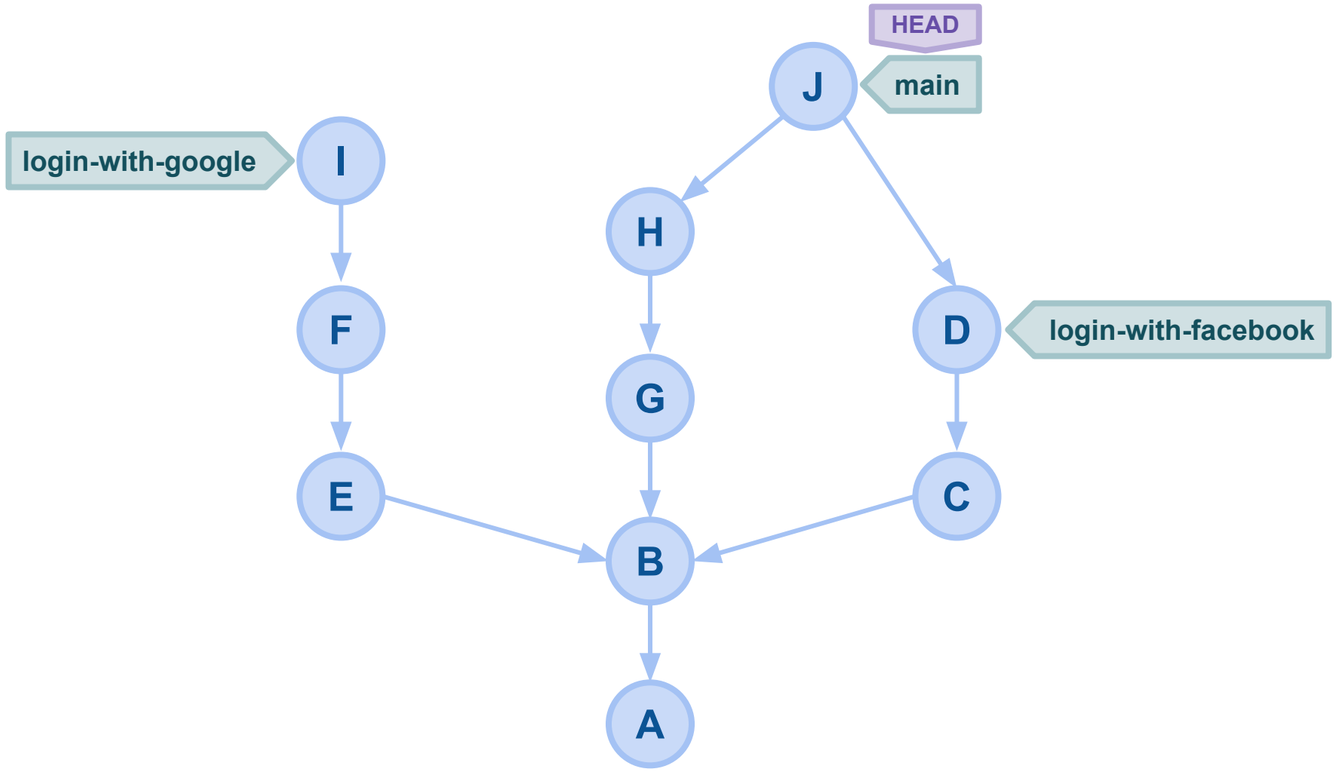
© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 🧐



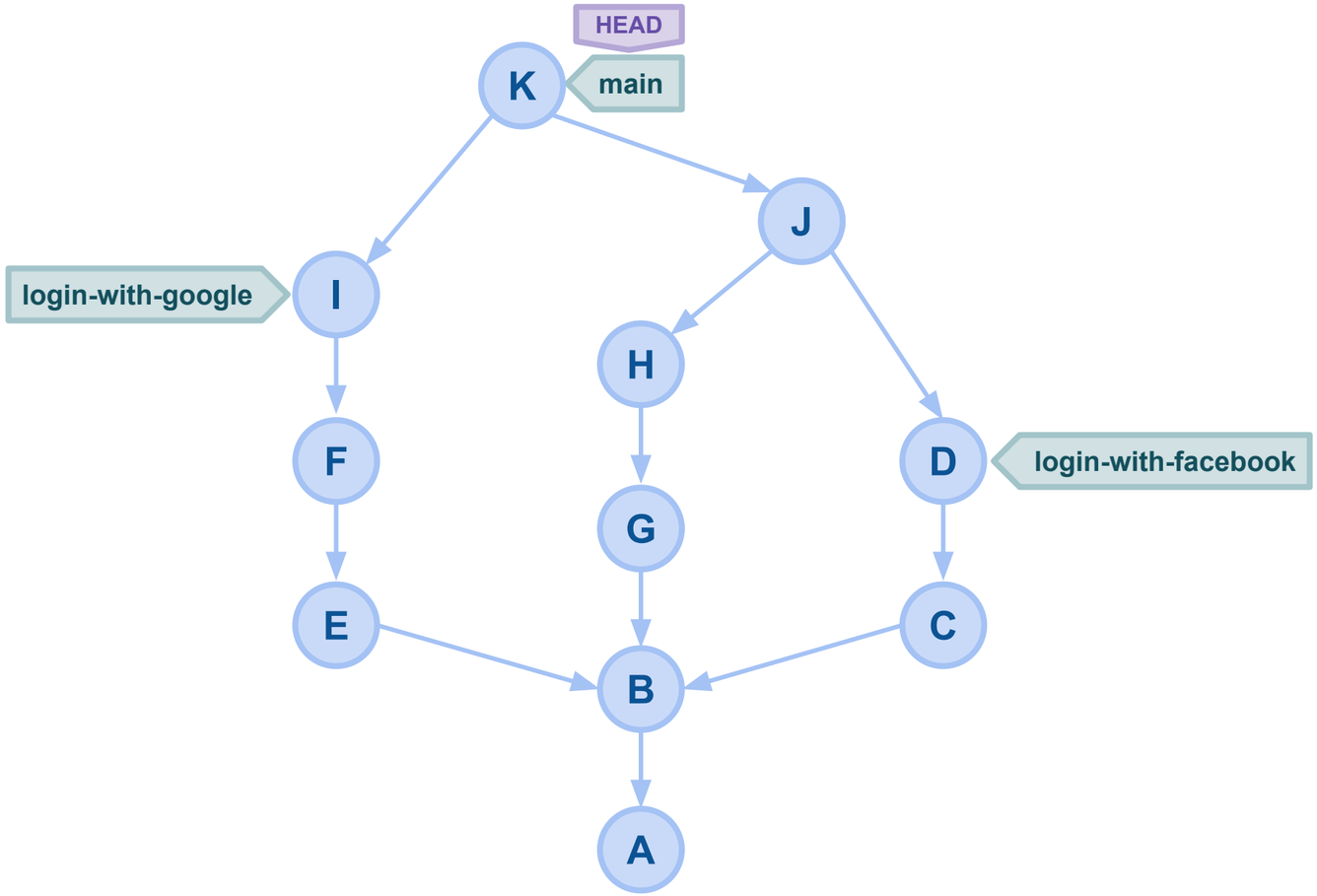
© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 🧐



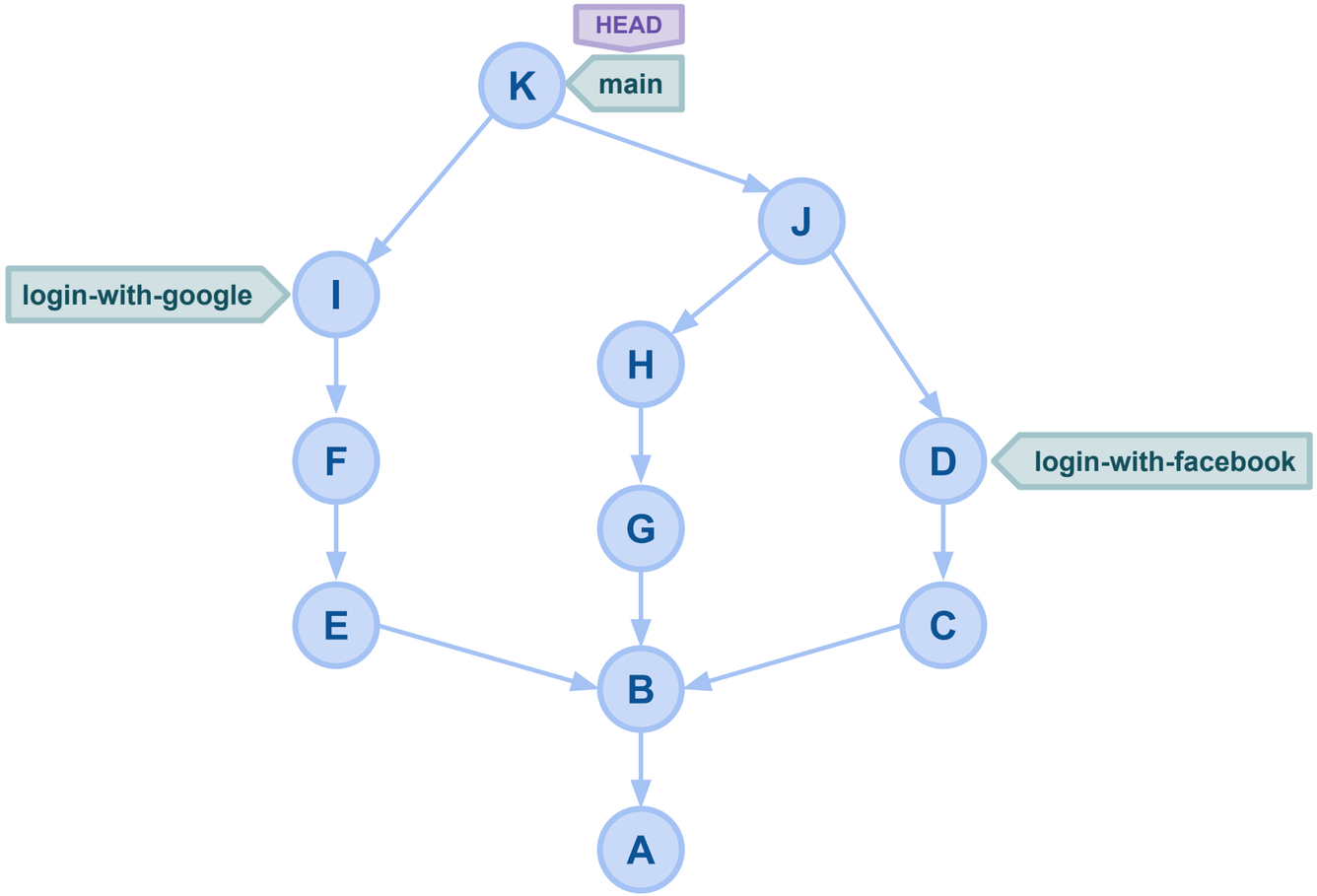
© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 🧐



© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 🧐



© All rights reserved. www.keepcoding.io

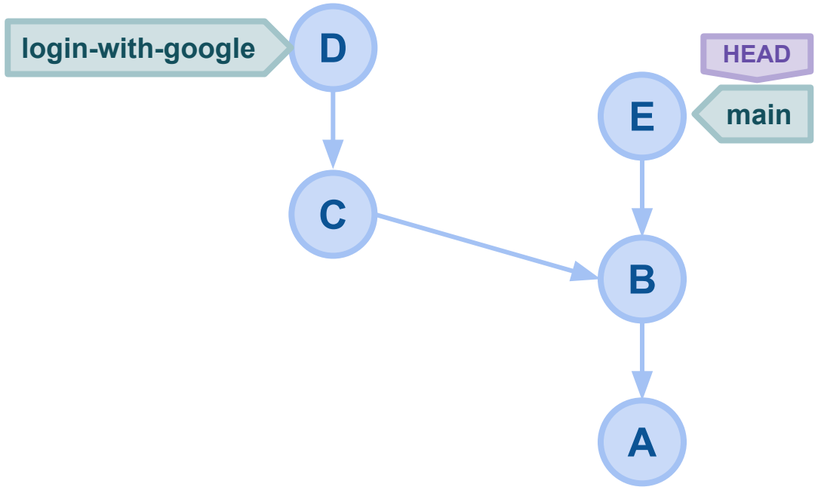
git merge

```
# Da acceso a los commits de la rama <branch name> en la rama en la que está HEAD
git merge <branch name>
```

© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 😇

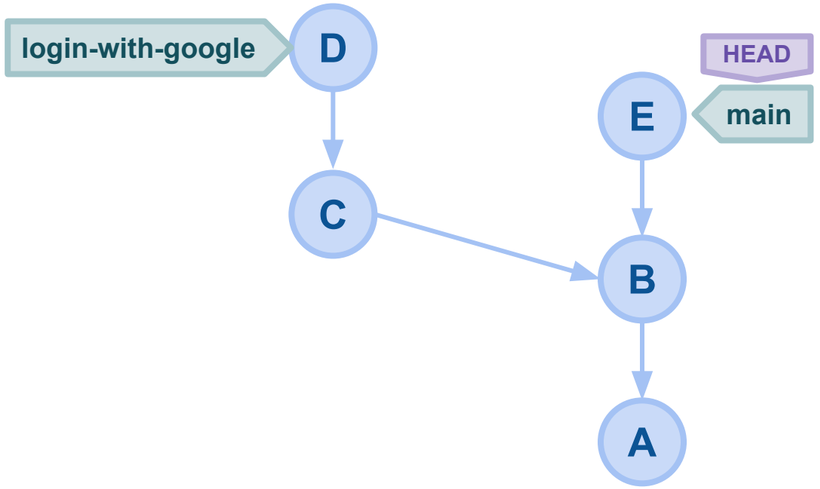
Incorporar el login con
Google a la rama main



© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 😇

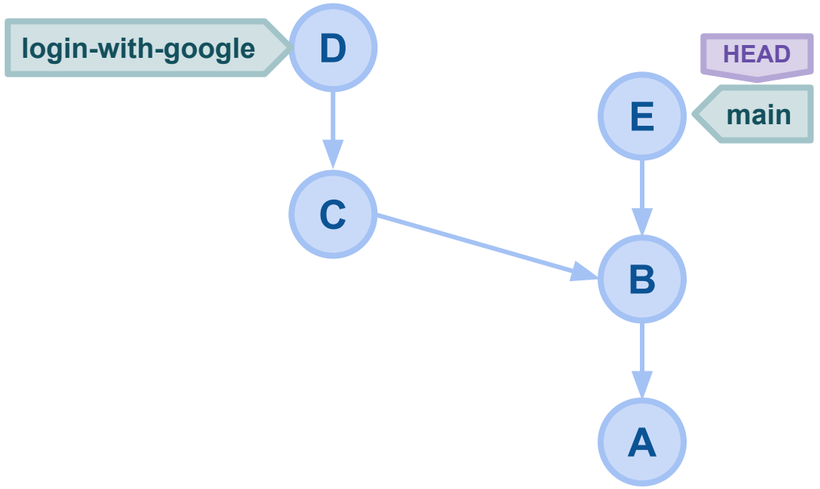
```
git merge login-with-google
```

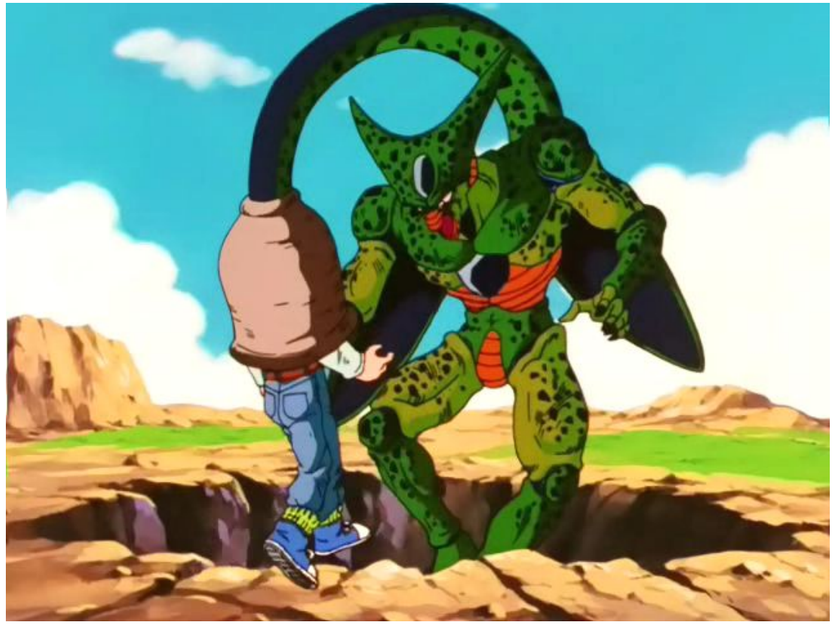


© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 😇

```
git merge login-with-google
```

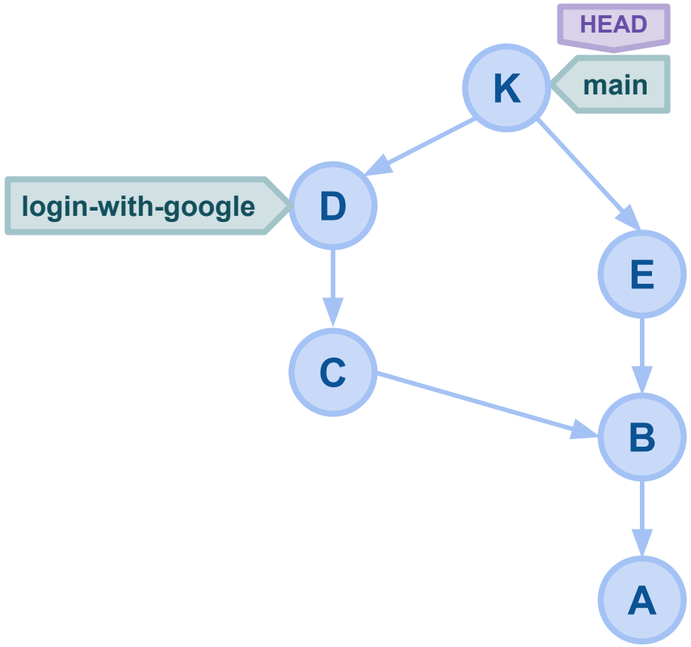




Regla nemotécnica: HEAD es Célula y absorbe la rama que indicas.

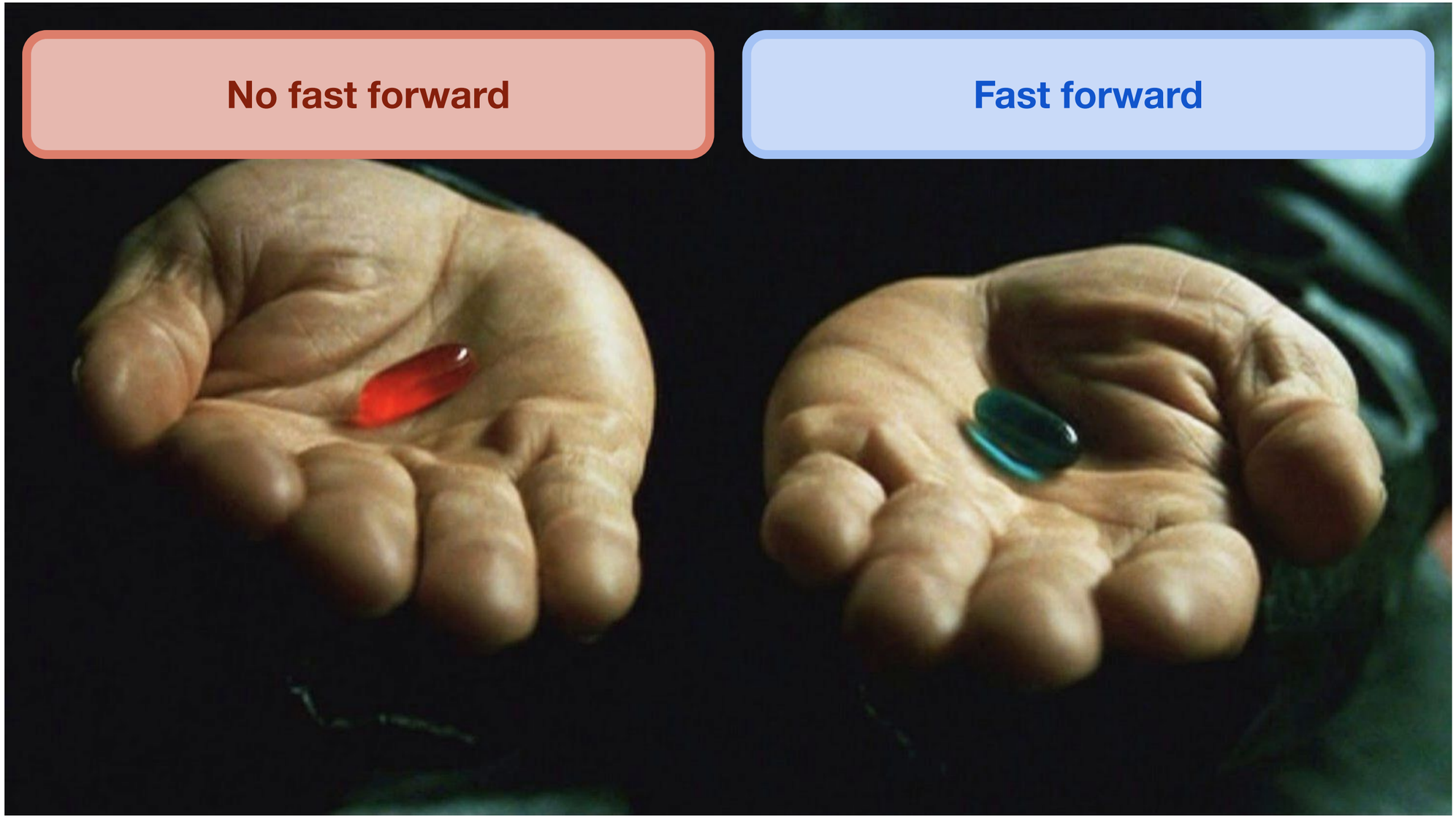
© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 😇



© All rights reserved. www.keepcoding.io

Los dos tipos de merge



© All rights reserved. www.keepcoding.io

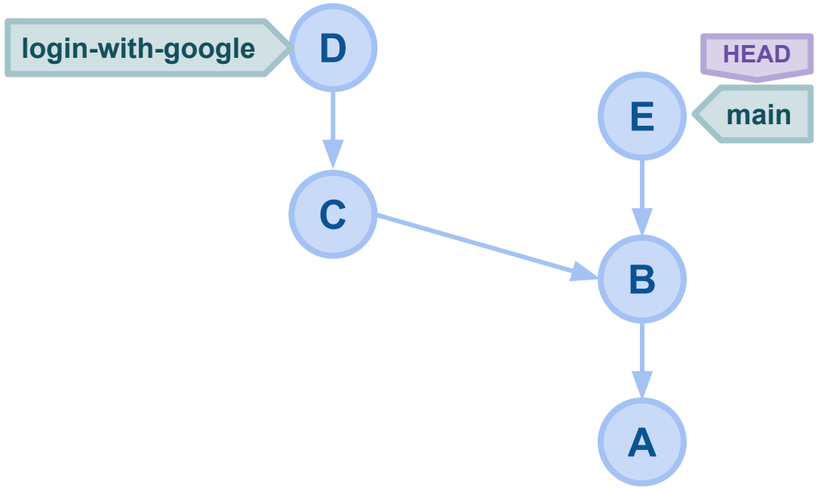
Los dos tipos de merge: no fast forward

No fast forward

© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 😌

No fast forward

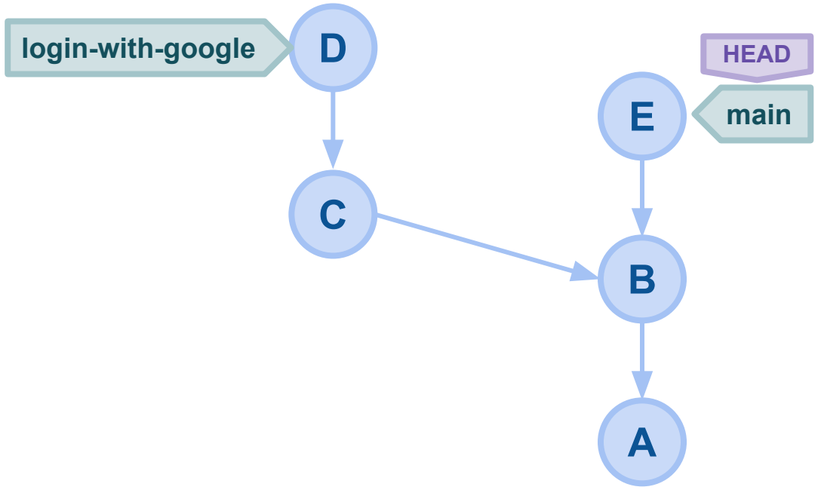


© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 😌

No fast forward

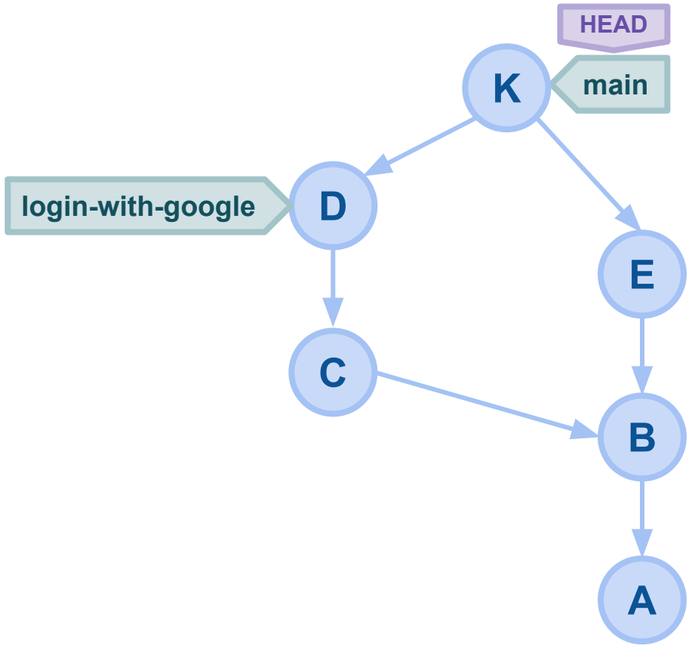
```
git merge login-with-google
```



© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 😌

No fast forward



© All rights reserved. www.keepcoding.io

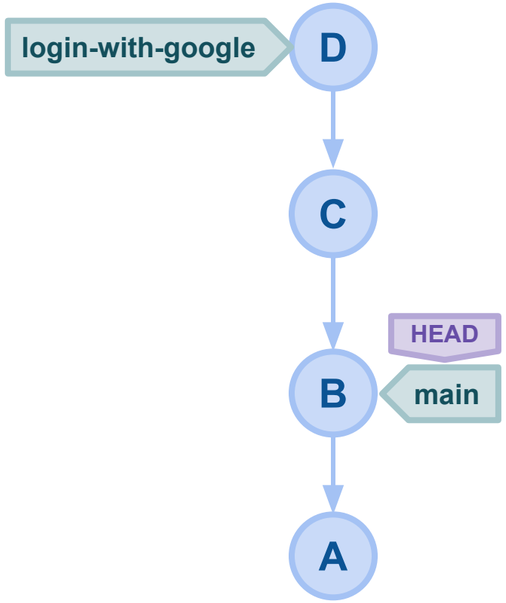
Los dos tipos de merge: fast forward

Fast forward

© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 🤨

Fast forward

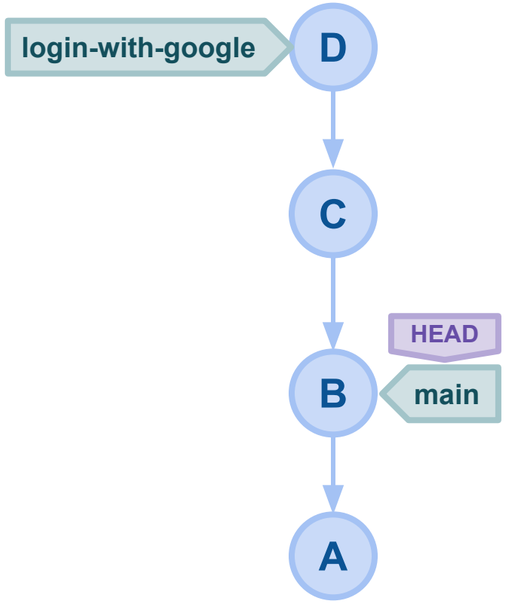


© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 🤨

Fast forward

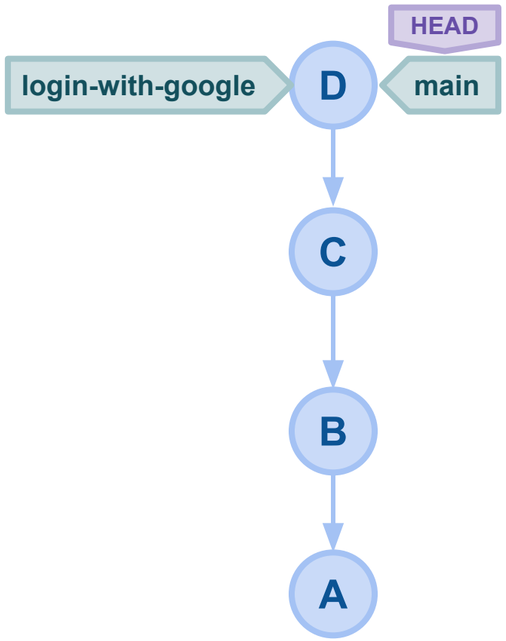
```
git merge login-with-google
```



© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 🤨

Fast forward

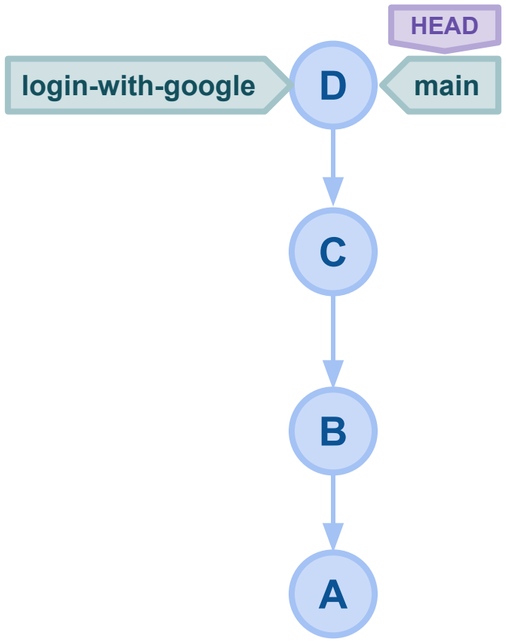


© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 🤨

Fast forward

Fast forward se basa en mover la rama en la que está HEAD hasta el commit de la otra rama



© All rights reserved. www.keepcoding.io

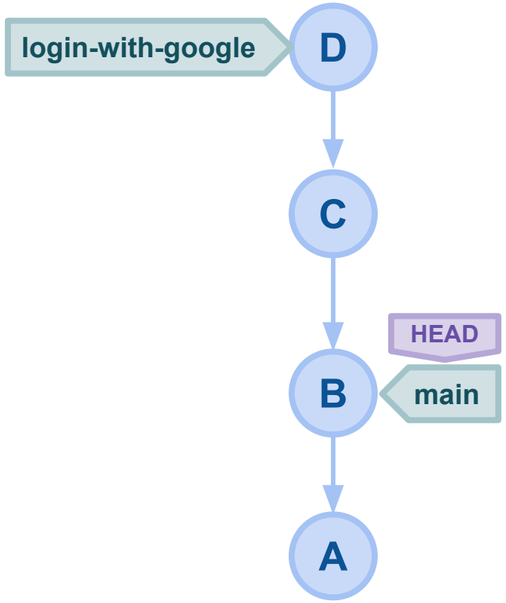
Los dos tipos de merge: fast forward

¿Puedo obligar a git a hacer
no fast forward?

© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 🤨

Fast forward

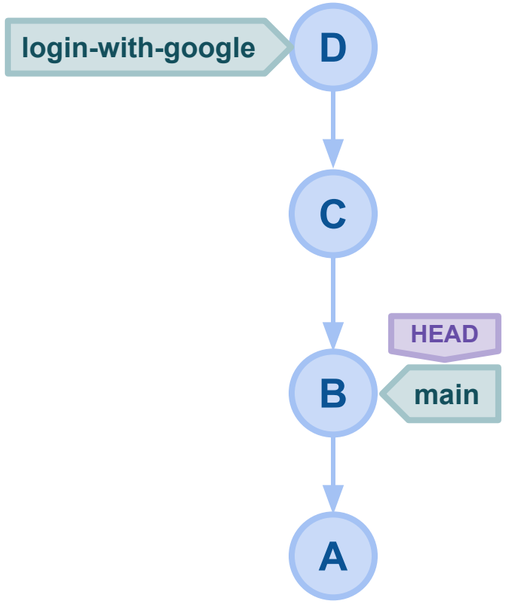


© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 🤨

Fast forward

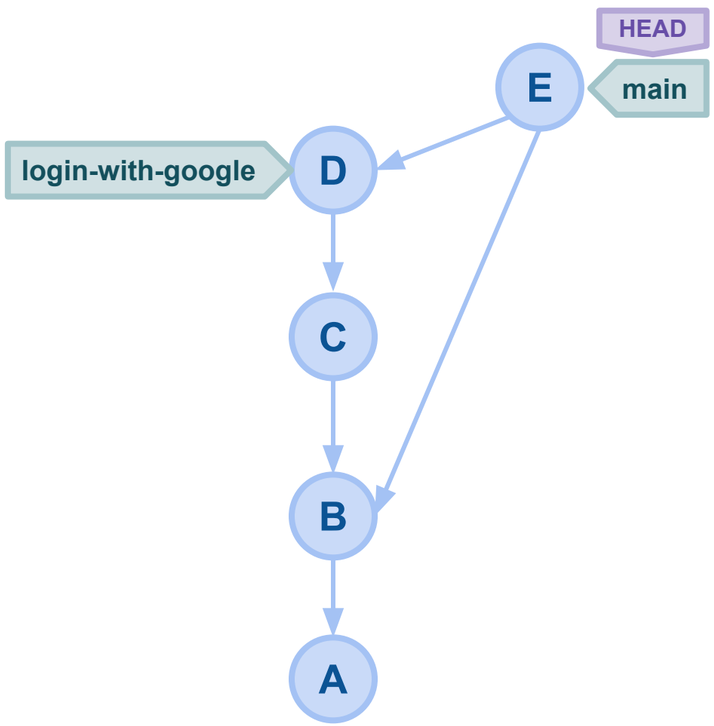
```
git merge --no-ff login-with-google
```



© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 🤨

Fast forward

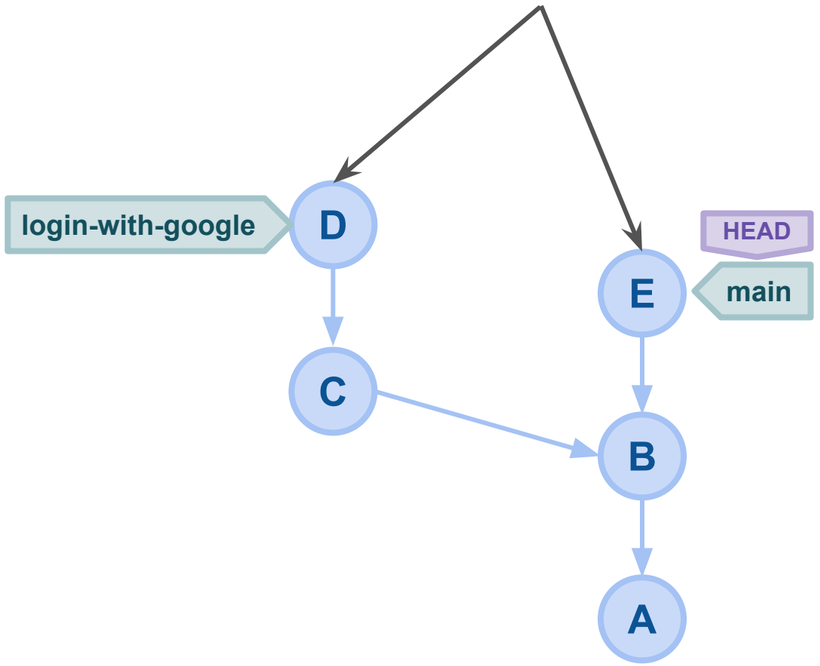


© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.73 😩

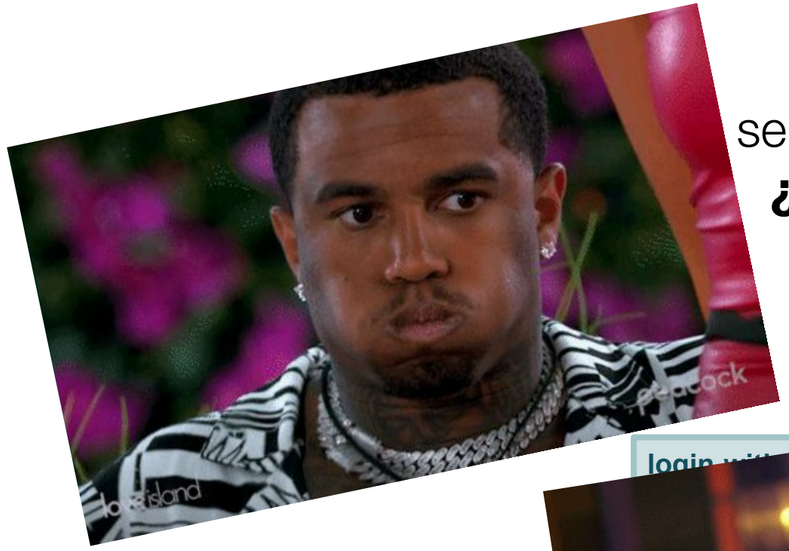
En ambos commits, se ha hecho un cambio en el archivo `index.js` en la línea 5.

¿Qué pasa si hacemos un merge?



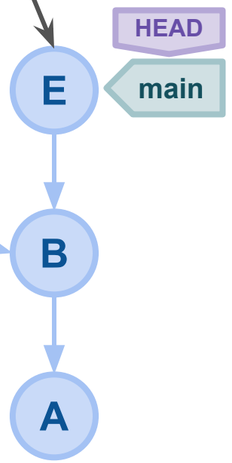
© All rights reserved. www.keepcoding.io

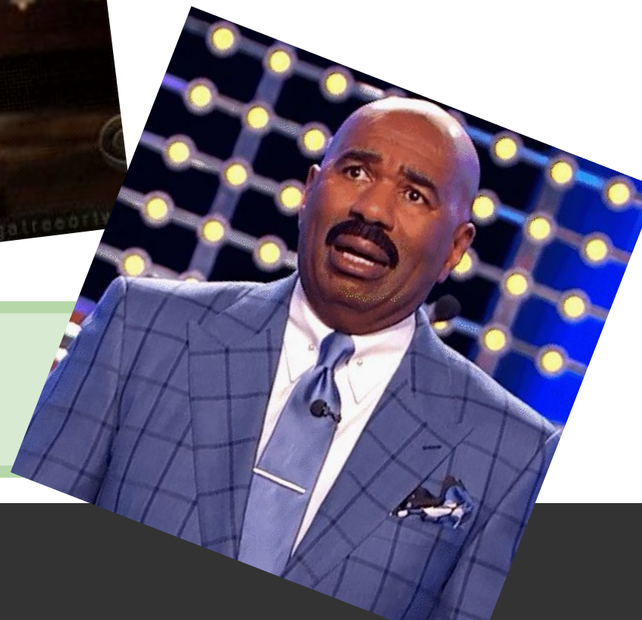
Git Simulator v.1.3.73 😩



se ha hecho un cambio en el archivo `index.j`

¿Qué pasa si hacemos un merge?





© All rights reserved. www.keepcoding.io

© All rights reserved. www.keepcoding.io

NO PROBLEMO

© All rights reserved. www.keepcoding.io

Conflictos

Cuando se monta el pollo

© All rights reserved. www.keepcoding.io

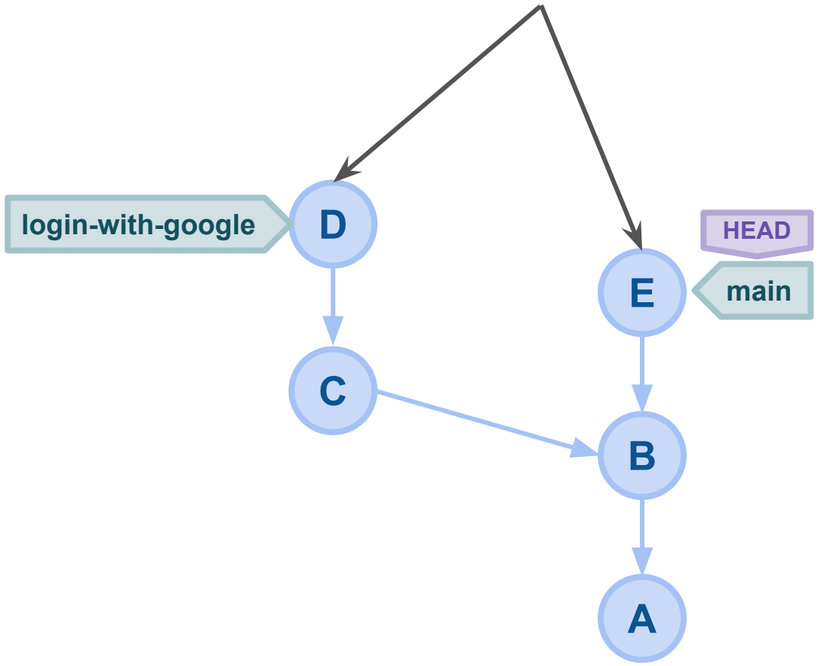
```
$ git merge master
```

```
Auto-merging war.txt
CONFLICT (content): Merge conflict in war.txt
Automatic merge failed; fix conflicts and then commit the result.
```

© All rights reserved. www.keepcoding.io

¿Cómo y cuándo se crea un conflicto?

Al hacer `merge` entre dos ramas diferentes (`main` y `login-with-google`) que contienen commits que hacen cambios en uno o varios archivos en las mismas líneas.



© All rights reserved. www.keepcoding.io

¿Cómo se solucionan?

- Un conflicto se produce al hacer un `merge` que <u>no puede ser fast forward</u>
- El conflicto sucede creando el commit que une las ramas. Se queda “a medias”
- Tenemos dos opciones:
  - Abortar el merge con `git merge --abort`
  - Solucionar los conflictos:
    i. Editando los archivos en conflicto (con nuestro editor de texto favorito)
    ii. Usar `git add <file>` en cada archivo que haya resuelto el conflicto
    iii. User `git commit` para finalizar el commit que une las dos ramas

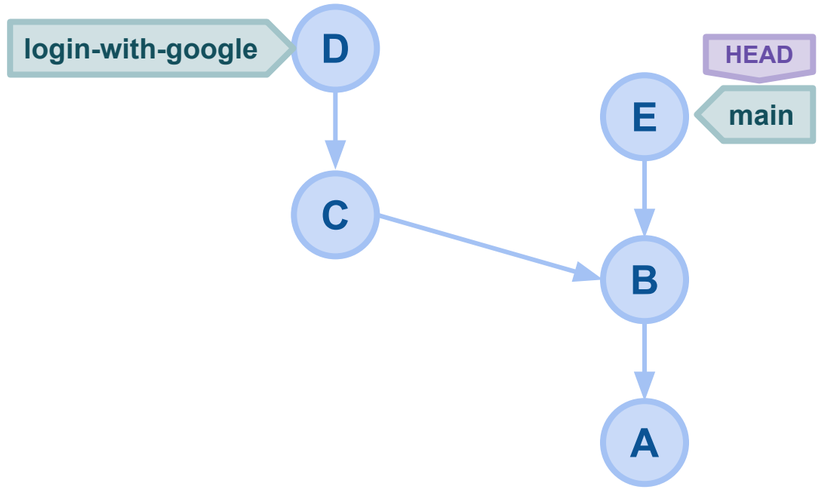
© All rights reserved. www.keepcoding.io

¿Qué pinta tienen los archivos en conflicto?

© All rights reserved. www.keepcoding.io

© All rights reserved. www.keepcoding.io

¿Qué pinta tienen los archivos en conflicto?



`index.js` en `main`

```
Roses are red
violets are #0000ff
all my base
Are belong to you.
```

`index.js` en `login-with-google`

```
Roses are red
Violets are blue,
All of my base
Are belong to you.
```

`index.js` en conflicto

```
Roses are red
<<<<<<<< HEAD
violets are #0000ff
all my base
=======
Violets are blue,
All of my base
>>>>>>>> BranchToMerge
Are belong to you.
```

© All rights reserved. www.keepcoding.io

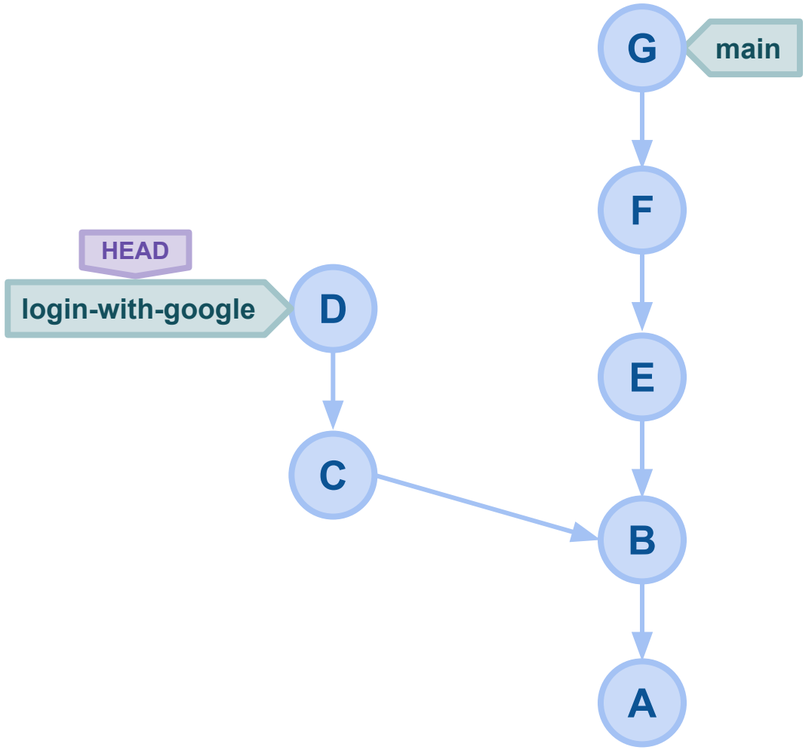
git cherry-pick

© All rights reserved. www.keepcoding.io

imgflip.com

Git Simulator v.1.3.74 😶

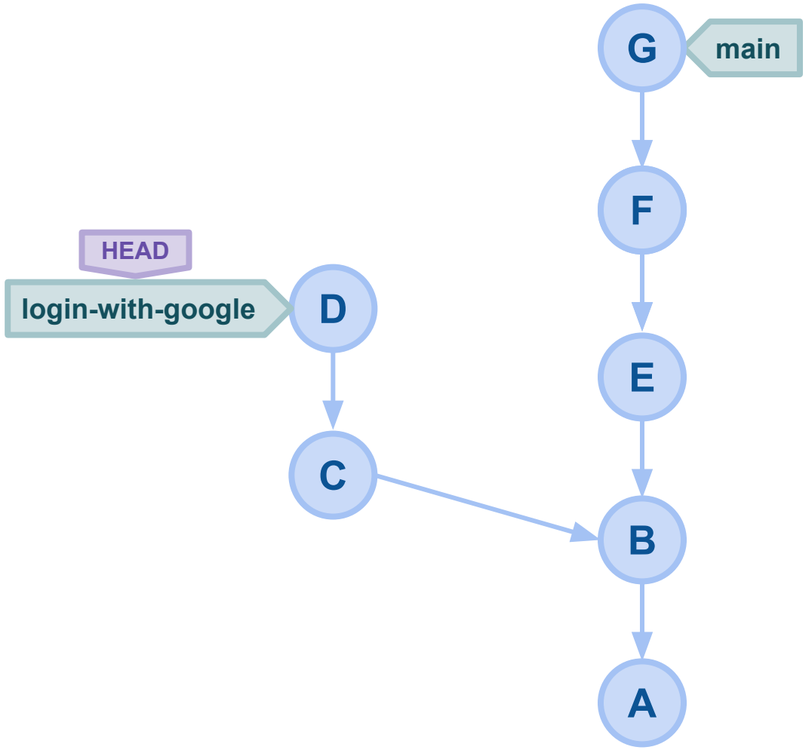
Necesito los cambios de **`F`** en **`login-with-google`**, pero no **`G ni E`**



© All rights reserved. www.keepcoding.io

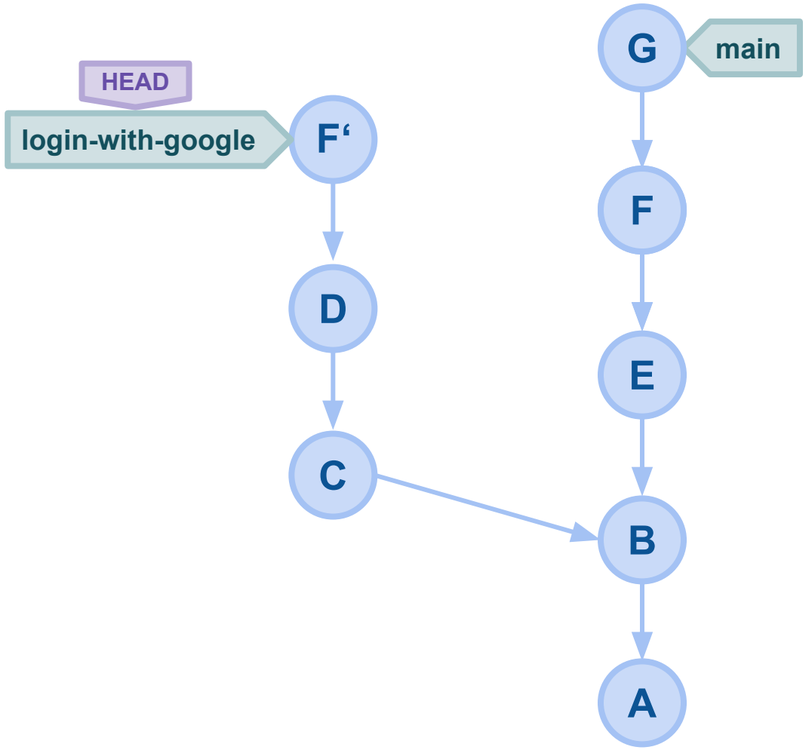
Git Simulator v.1.3.74 😶

```
git cherry-pick F
```



© All rights reserved. www.keepcoding.io

Git Simulator v.1.3.74 😶



© All rights reserved. www.keepcoding.io

Repos remotos

Júntate con los demás frikis en GitHub

© All rights reserved. www.keepcoding.io

¿Qué es GitHub?

- Plataforma para alojar proyectos con Git y trabajo colaborativo
- Infinidad de funcionalidades
- Es **GRATUITA** (modelo freemium)
- Con páginas de 404 not found y 500 server error muy chulas
- El Open Source creció gracias a sus forks y pull requests
- Su logo es un "octogato" (Octocat)

© All rights reserved. www.keepcoding.io

git clone

```
# Clona el repositorio alojado en <repo-url> en una carpeta con su mismo nombre
git clone <repo-url>

# Clona el repositorio alojado en <repo-url> en la carpeta <folder>
git clone <repo-url> <folder>
```

© All rights reserved. www.keepcoding.io

¿Y si he creado el repo en mi ordenador?

© All rights reserved. www.keepcoding.io

git remote

```
# Ver los repositorios remoto definidos en el repo que estoy trabajando
git remote

# Añadir el repositorio remoto con nombre <repo-name> y url <repo-url> en el repo que estoy trabajando
git remote add <repo-url> <repo-name>

# Elimina el repositorio remoto con nombre <repo-name> del repo que estoy trabajando
git remote remove <repo-name>
```

© All rights reserved. www.keepcoding.io

¿Cómo funcionan los repos remotos?

© All rights reserved. www.keepcoding.io

¿Cómo funcionan los repos remotos?

© All rights reserved. www.keepcoding.io

¿Cómo funcionan los repos remotos?

¿Sabes trabajar con ramas?
Entonces también con repos remotos.

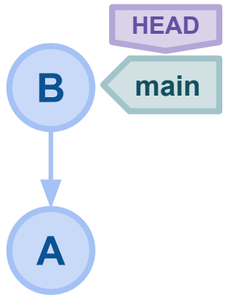
Los repos remotos usan “ramas remotas”, que funcionan igual que las ramas normales.

Esas “ramas remotas”, son las ramas en las que trabajan otros miembros del equipo (que son los que te generan conflictos).

© All rights reserved. www.keepcoding.io

© All rights reserved. www.keepcoding.io

Git Simulator v.1.4.0 🤪



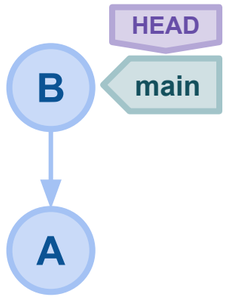
remote

local

© All rights reserved. www.keepcoding.io

Git Simulator v.1.4.0 🤪

```
git clone user@remote/repo.git
```

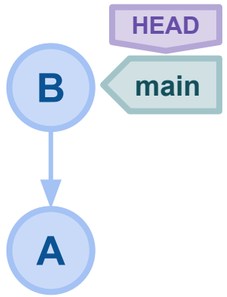


remote

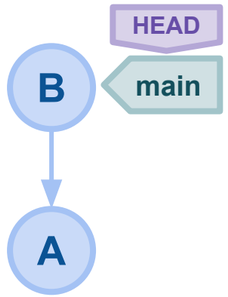
local

© All rights reserved. www.keepcoding.io

Git Simulator v.1.4.0 🤪



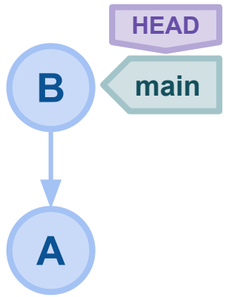
remote



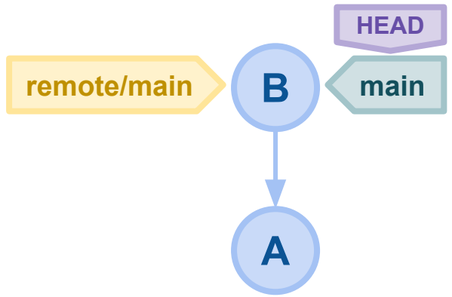
local

© All rights reserved. www.keepcoding.io

Git Simulator v.1.4.0 🤪



remote

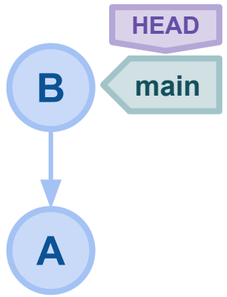


local

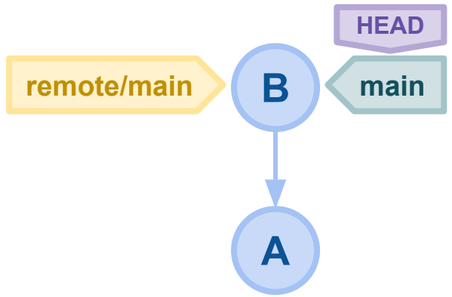
© All rights reserved. www.keepcoding.io

Git Simulator v.1.4.0 🤪

```
git commit -m “Añado commit C”
```



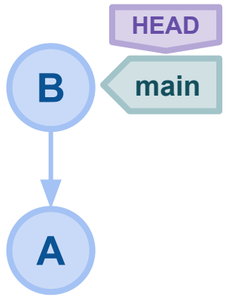
remote



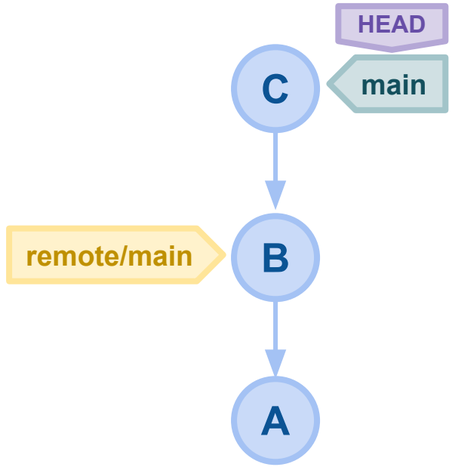
local

© All rights reserved. www.keepcoding.io

Git Simulator v.1.4.0 🤪



remote

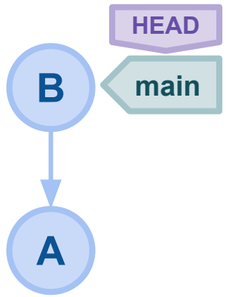


local

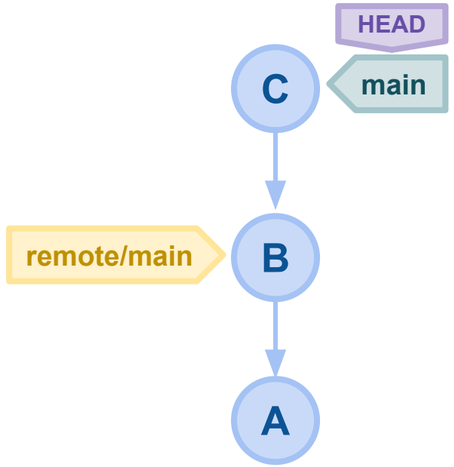
© All rights reserved. www.keepcoding.io

Git Simulator v.1.4.0 🤪

```
git push
```



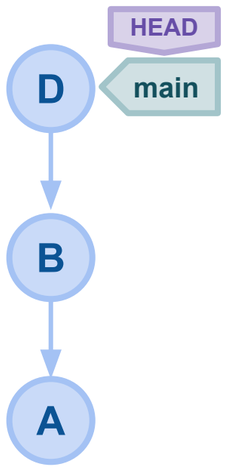
remote



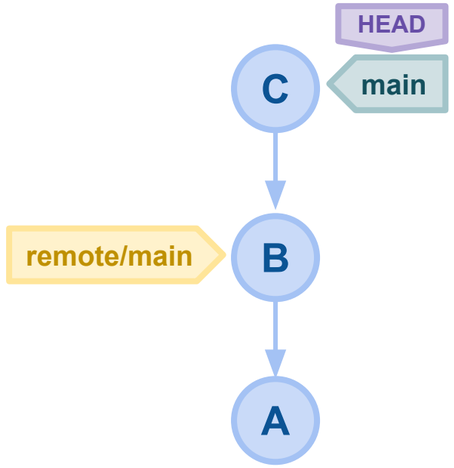
local

© All rights reserved. www.keepcoding.io

Git Simulator v.1.4.0 🤪



remote

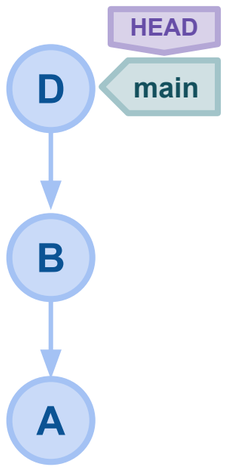


local

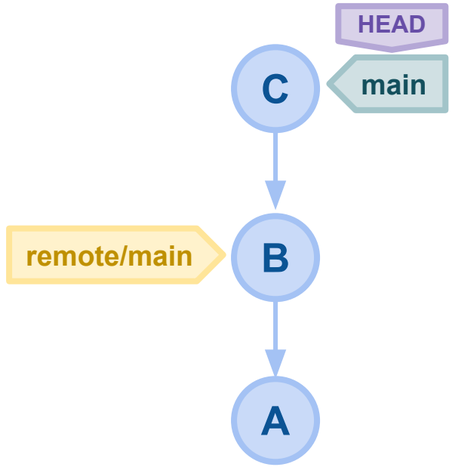
© All rights reserved. www.keepcoding.io

Git Simulator v.1.4.0 🤪

```
git push remote main
```



remote

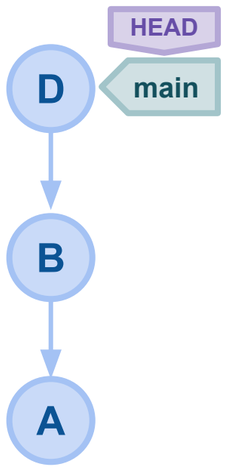


local

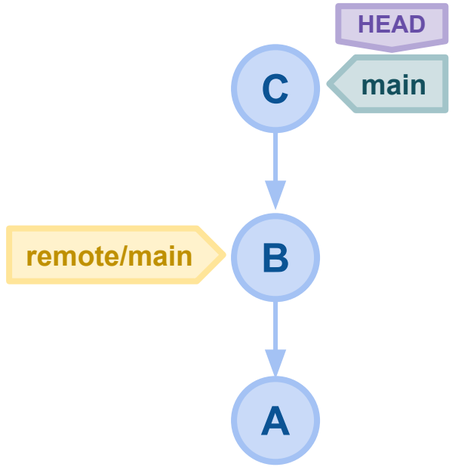
© All rights reserved. www.keepcoding.io

Git Simulator v.1.4.0 🤪

```
! [rejected] main -> main
error: failed to push some refs to remote
```



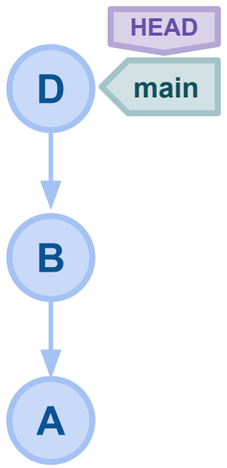
remote



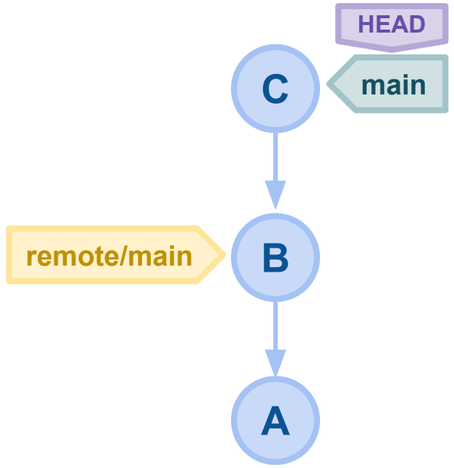
local

© All rights reserved. www.keepcoding.io

Git Simulator v.1.4.0 🤪



remote

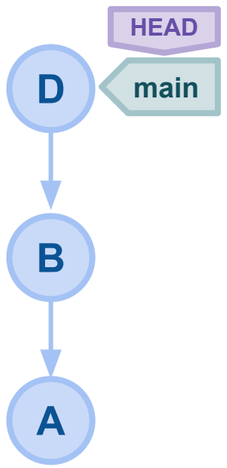


local

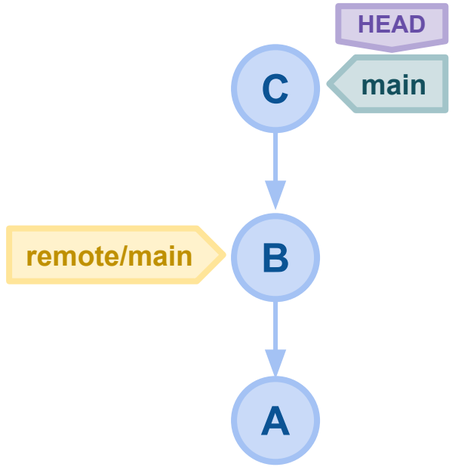
© All rights reserved. www.keepcoding.io

Git Simulator v.1.4.0 🤪

```
git fetch
```



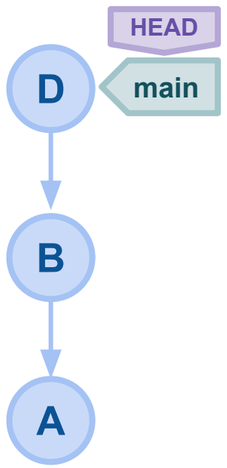
remote



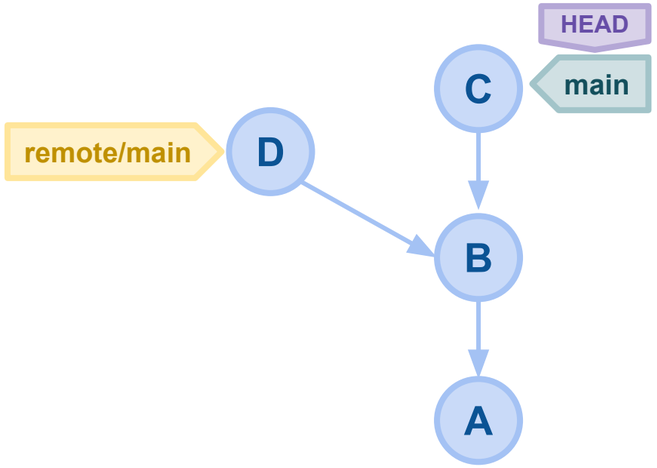
local

© All rights reserved. www.keepcoding.io

Git Simulator v.1.4.0 🤪



remote

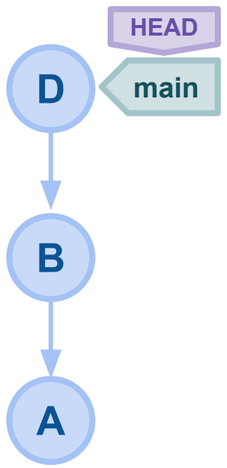


local

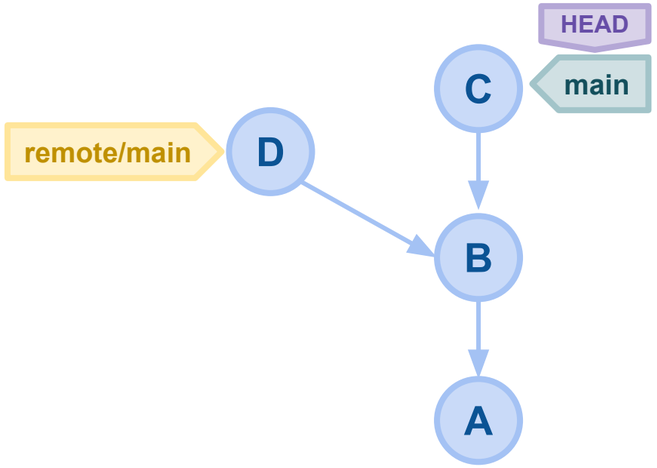
© All rights reserved. www.keepcoding.io

Git Simulator v.1.4.0 🤪

```
git merge remote/main
```



remote

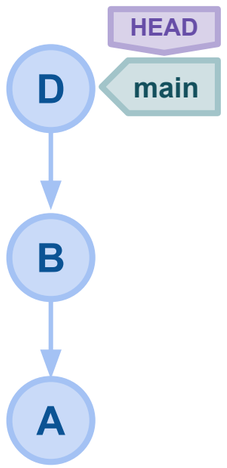


local

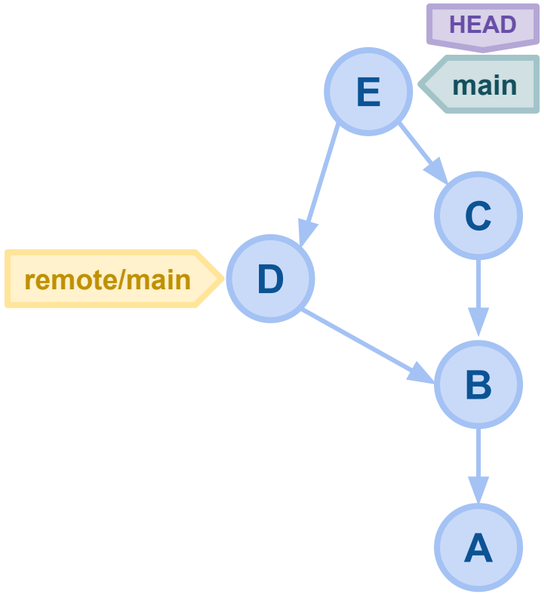
© All rights reserved. www.keepcoding.io

Git Simulator v.1.4.0 🤪

```
git push
```



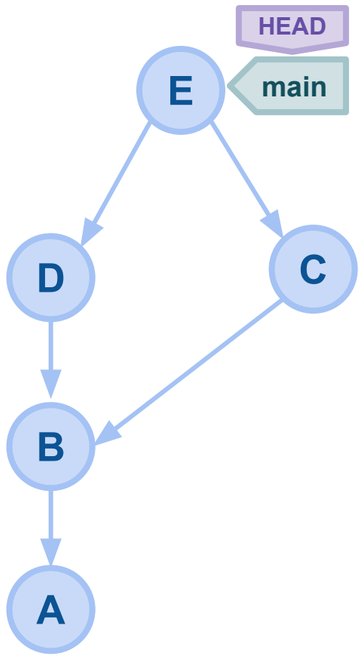
remote



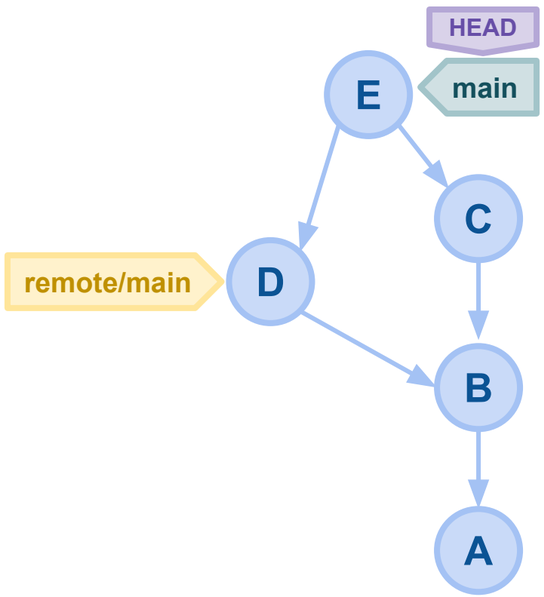
local

© All rights reserved. www.keepcoding.io

Git Simulator v.1.4.0 🤪



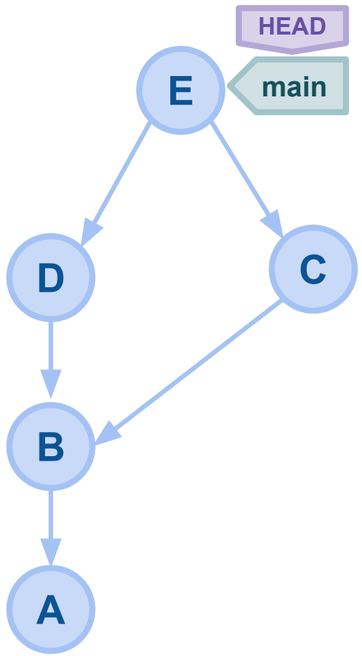
remote



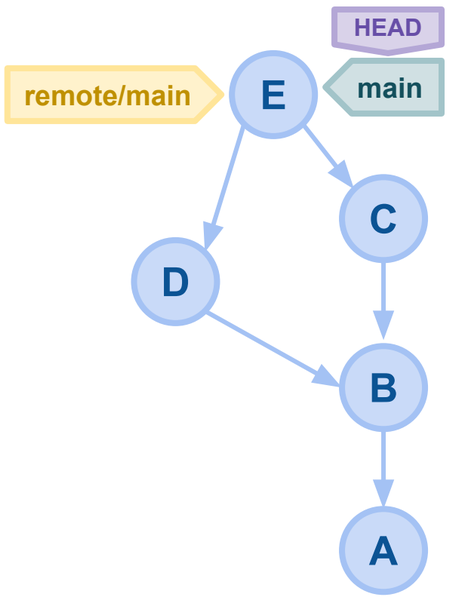
local

© All rights reserved. www.keepcoding.io

Git Simulator v.1.4.0 🤪



remote

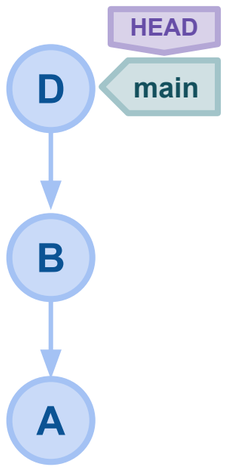


local

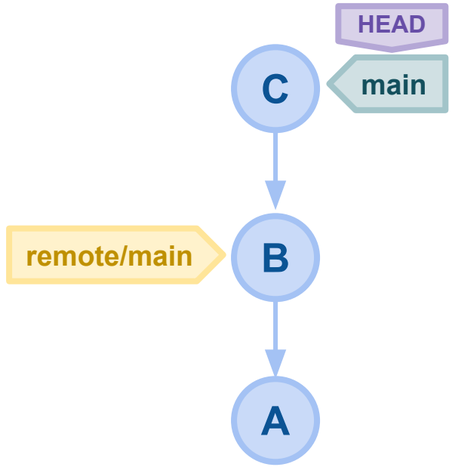
© All rights reserved. www.keepcoding.io

Git Simulator v.1.4.0 🤪

```
! [rejected] main -> main
error: failed to push some refs to remote
```



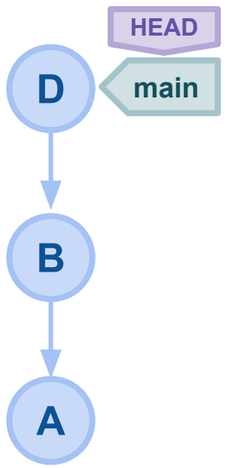
remote



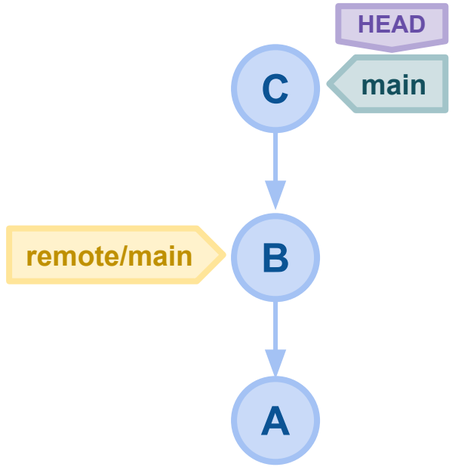
local

© All rights reserved. www.keepcoding.io

Git Simulator v.1.4.0 🤪



remote

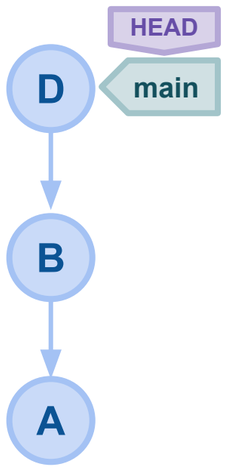


local

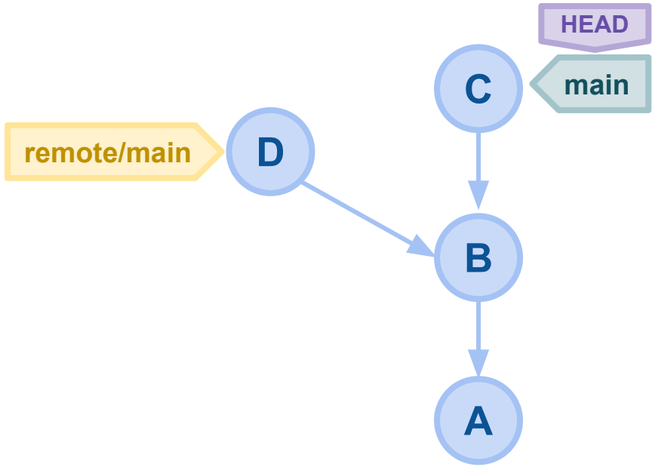
© All rights reserved. www.keepcoding.io

Git Simulator v.1.4.0 🤪

```
git pull
```



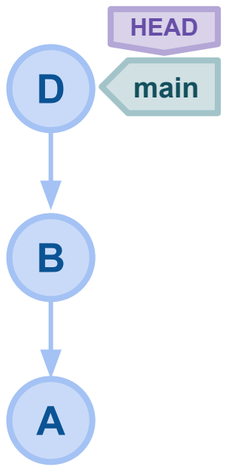
remote



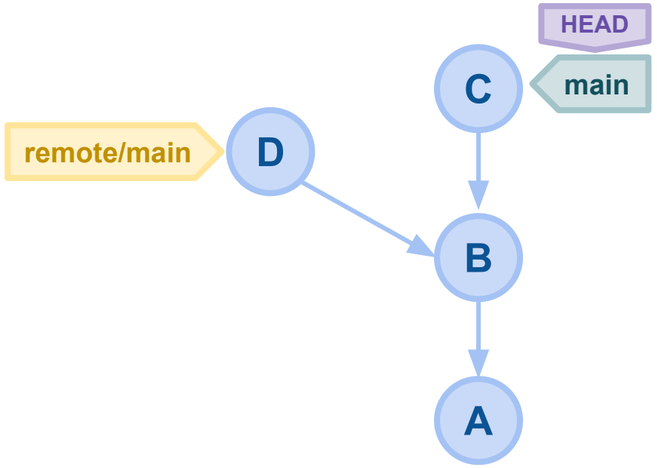
local

© All rights reserved. www.keepcoding.io

Git Simulator v.1.4.0 🤪



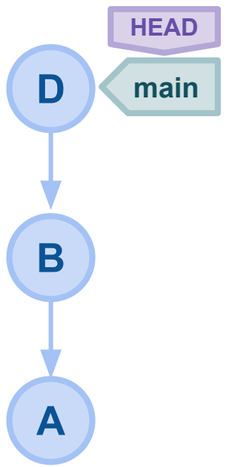
remote



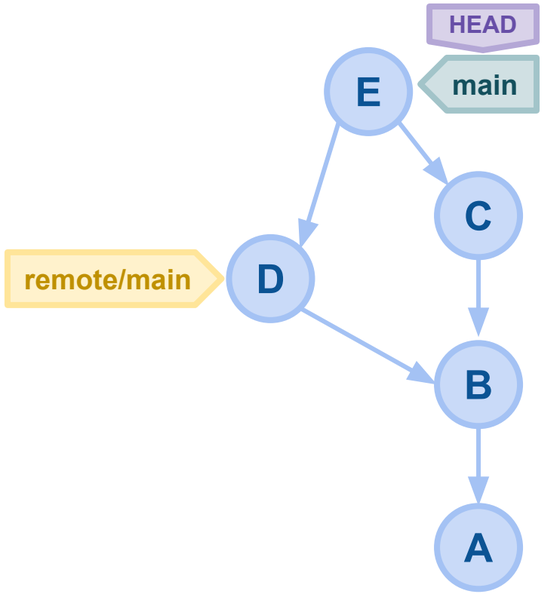
local

© All rights reserved. www.keepcoding.io

Git Simulator v.1.4.0 🤪



remote



local

© All rights reserved. www.keepcoding.io

¿Qué ha hecho `git pull`?

© All rights reserved. www.keepcoding.io

```
git fetch + git merge = git pull
```

© All rights reserved. www.keepcoding.io

Repos remotos

```
# Sube los cambios de la rama en la que está HEAD a su remote por defecto
git push

# Sube los cambios del <branch/tag> al remote <remote>
git push <remote> <branch/tag>

# Sube los cambios y establece el remote por defecto
git push -u <remote> <branch/tag>

# Fuerza a reestablecer la rama remota al mismo commit que en la local
git push -f

# Descarga cambios de todas las ramas y tags del repo remoto
git fetch

# Hace un fetch y mergea la rama en la que estoy con su rama remota
git pull
```

© All rights reserved. www.keepcoding.io

Forks, pull requests & code review

- Un **fork** es una copia de un repositorio en tu cuenta
  - Por tanto, como está en tu cuenta, puedes modificar

- Un **pull request** es una solicitud para *mergear* entre dos ramas
  - Las dos ramas pueden ser del mismo repo o de repos distintos
  - En el pull request, se pueden hacer code reviews

© All rights reserved. www.keepcoding.io

¿Cómo colaborar con un proyecto Open Source?

1. Hacemos un fork del proyecto
2. Como el fork está en nuestra cuenta, podemos hacer cambios
3. Una vez hechos los cambios, podemos hacer un pull request para solicitar que incorporen el código del fork al del repo principal (si lo consideran oportuno)

© All rights reserved. www.keepcoding.io

La carpeta `.git` y archivos `.git*`

© All rights reserved. www.keepcoding.io

La carpeta `.git`

- La carpeta `.git` es donde Git almacena toda la info del repo
- `objects/` - donde se almacena todo (commits, trees, blobs…)
- `refs/` - ficheros con el commit al que apuntan ramas/tags
- `index` - información de estado de staging area
- `config` - archivo de configuración del repositorio
- `hooks` - scripts que nos permiten automatizar tareas

© All rights reserved. www.keepcoding.io

¿Has dicho
automatizar?

Hooks

- Podemos usar cualquier lenguaje
- Se ejecutan en puntos específicos del workflow
  - `pre-commit` - se ejecuta antes de cada commit
  - `commit-msg` - para validar mensajes de commit
  - `pre-push` - se ejecuta antes de cada push
  - `post-receive` - se ejecuta antes de recibir un push (server)
  - `pre-receive` - se ejecuta después de recibir un push (server)
  - ...

© All rights reserved. www.keepcoding.io

HEAD, ORIG_HEAD, FETCH_HEAD

- `HEAD` - contiene donde apunta el puntero HEAD
- `ORIG_HEAD` - backup de HEAD antes de operaciones peligrosas
- `FETCH_HEAD` - donde apuntaba la rama en el repo remoto la última vez que hicimos `git fetch` (usado durante `pull` para hacer el `merge`)

© All rights reserved. www.keepcoding.io

El archivo `.gitignore`

Sirve para **indicar a Git qué archivos queremos que ignore** aunque estén presentes en nuestro working copy:

- Archivos con credenciales (sí, los `.env`)
- Archivos de logs, bases de datos SQLite
- Dependencias (⚠️⚠️ no hagáis `commit` de `node_modules` ⚠️⚠️)
- Cualquier archivo que se pueda generar desde el código

https://gitignore.io

© All rights reserved. www.keepcoding.io

Ejemplo .gitignore

```
# macOS
.DS_Store

# Windows
Thumbs.db

# Linux
*~

# Python
__pycache__/

# Node
node_modules/

# General
*.log
*.sqlite

# Excepciones
.vscode/
!.vscode/launch.json
```

© All rights reserved. www.keepcoding.io

El archivo `.gitattributes`

Indica a Git cómo debe manejar cierto tipos de archivos:

- Conversión de fin de línea (CRLF/LF)
- Text encoding
- Estrategias de mergeo
- Detección de binarios
- Git LFS (Large File Storage) tracking

© All rights reserved. www.keepcoding.io

Ejemplo .gitattributes

```
# Binary files
*.jpg binary
*.png binary
*.exe binary

# Programming files with specific handling
*.js text eol=lf
*.css text eol=lf
*.html text eol=lf

# Force specific files to be treated as text
README text
Makefile text

# Use Git LFS for large files
*.psd filter=lfs diff=lfs merge=lfs -text
*.zip filter=lfs diff=lfs merge=lfs -text
```

https://git-scm.com/docs/gitattributes

© All rights reserved. www.keepcoding.io

El archivo `.gitkeep`

Se utilizan para mantener en Git directorios vacíos.

(sí, el típico `uploads/` que si no existe falla cuando subes un archivo)

No es del estándar de Git, sino que es una convención de la comunidad.

```
$ touch uploads/.gitkeep
$ git add uploads/.gitkeep
$ git commit -m “para que no fallen los uploads”
```

© All rights reserved. www.keepcoding.io

Cuando todo va mal

© All rights reserved. www.keepcoding.io

© All rights reserved. www.keepcoding.io

`git gc`

¿Git empieza a ir lento? ¿`.git` ocupa mucho espacio?

Otimiza el repo, elimina lo innecesario con el Garbage Collector

- `git gc`
- `git gc --aggresive` - más lento, pero más minucioso
- `git gc --auto` - Git decide el mejor modo

© All rights reserved. www.keepcoding.io

`git fsck`

¿Git hace cosas raras? ¿Se colgó el sistema mientras Git trabajaba?

Verifica la integridad del repo comprobando los objetos y refs.

- `git fsck`
- `git fsck --full`
- `git fsck --unreachable` - Muestra objetos inalcanzables
- `git fsck --cache --unreachable --dangling --root --tags`

  - Comprobación a fondo, incluso reflogs

© All rights reserved. www.keepcoding.io

Submodules

© All rights reserved. www.keepcoding.io

Submódulos

Permite incluir un repo de Git como un subdirectorio dentro de un repo.

Muy útil para:

- Compartir librerías o scripts en múltiples proyectos
- Usar librerías de terceros en versiones específicas no publicadas
- Proyectos grandes con componentes independientes

https://git-scm.com/book/en/v2/Git-Tools-Submodules

© All rights reserved. www.keepcoding.io

Submódulos

```
git submodule add <url> <folder>
```

```
git submodule add https://github.com/PHPMailer libs/mailer
```

Esto crea un archivo `.gitmodules` que debe formar parte del proyecto.

© All rights reserved. www.keepcoding.io

© All rights reserved. www.keepcoding.io

Submódulos

Trabajar con submódulos implica trabajo extra:

- Cuando clonas un repo con submódulos, no se inicializan sólos
- Al entrar en la carpeta del submódulo, trabajas en “otro repo”
- Al hacer pull desde el repo principal, no se actualizan los submódulos

© All rights reserved. www.keepcoding.io

Clonar e inicializar submódulos

```
$ git clone --recursive <url>
```

```
# Por separado
$ git clone <url>
$ git submodule init
$ git submodule update
```

© All rights reserved. www.keepcoding.io

Estado de submódulos

```
$ git status submodules

# Por separado
$ cd lib/mailer
$ git status
```

© All rights reserved. www.keepcoding.io

Actualizar submódulos a última versión

```
$ git submodule update --remote --merge
```

```
# Por separado
$ cd lib/mailer
$ git pull
```

© All rights reserved. www.keepcoding.io

Eliminar submódulo

```
$ git submodule deinit -f <path>
$ git rm -f <path>
$ rm -rf .git/modules/<path>
$ git commit -m "Remove submodule"
```

© All rights reserved. www.keepcoding.io

¿Por dónde sigo?

- Aprende más Git jugando: Learn Git Branching, Oh My Git!
- Aprende algunos flujos de trabajo, como Gitflow o Gitlab Flow
- Aprende GitHub Actions para automatizar cosas cuando hagas push
- En unos meses, aprende `git rebase`

© All rights reserved. www.keepcoding.io

GRACIAS

www.keepcoding.io

© All rights reserved. www.keepcoding.io